

Journal Pre-proof

Programming on Bitcoin: A Survey of Layer 1 and Layer 2 Technologies in Bitcoin Ecosystem

Guofu Liao, Taotao Wang, Long Shi, Jiqun Zhang, Yihan Xia et al.

PII: S2096-7209(25)00135-6
DOI: <https://doi.org/10.1016/j.bcra.2025.100408>
Reference: BCRA 100408

To appear in: *Blockchain: Research and Applications*

Received date: 8 April 2025
Revised date: 5 August 2025
Accepted date: 5 September 2025

Please cite this article as: G. Liao, T. Wang, L. Shi et al., Programming on Bitcoin: A Survey of Layer 1 and Layer 2 Technologies in Bitcoin Ecosystem, *Blockchain: Research and Applications*, 100408, doi: <https://doi.org/10.1016/j.bcra.2025.100408>.

This is a PDF file of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability, but it is not yet the definitive version of record. This version will undergo additional copyediting, typesetting and review before it is published in its final form, but we are providing this version to give early visibility of the article. Please note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2025 Published by Elsevier.



Programming on Bitcoin: A Survey of Layer 1 and Layer 2 Technologies in Bitcoin Ecosystem

Guofu Liao^a, Taotao Wang^{a,*}, Long Shi^b, Jiqun Zhang^a, Yihan Xia^a, Qing Yang^a, Xiaoxiao Wu^a and Shengli Zhang^a

^aCollege of Electronics and Information Engineering, Shenzhen University, Shenzhen 518060, China

^bSchool of Electronic and Optical Engineering, Nanjing University of Science and Technology, Nanjing 210094, China

ARTICLE INFO

Keywords:

Bitcoin
Bitcoin programming
Taproot
Bitcoin Layer 2 protocols
Blockchain

ABSTRACT

This paper surveys the innovative protocols designed to enhance the programming functionality of the Bitcoin blockchain, collectively forming a significant component of the esteemed "Bitcoin Ecosystem". While Bitcoin was originally designed for peer-to-peer payments using a basic scripting language, developers have found ways to significantly expand its programming capabilities. A key milestone in this evolution is the 2021 Taproot upgrade, which introduced new cryptographic tools that made Bitcoin more private and programmable. This upgrade enabled the development of new protocols for digital assets, such as NFTs and fungible tokens, which have significantly broadened Bitcoin's capabilities beyond basic payments. At the same time, a new series of protocols has emerged within the Bitcoin ecosystem, which we categorize as Layer 2 solutions akin to Ethereum's Layer 2. These protocols further enhance Bitcoin's performance and programming capability. Although prior studies have explored Bitcoin's programming capability, there remains a lack of comprehensive review and empirical analysis of ecosystem protocols. This research bridges this gap and offers valuable insights for both practitioners and researchers alike.

1. Introduction

Blockchain technology was originally proposed to create a decentralized, trustless, and transparent transaction system. Bitcoin, initially proposed in 2008 by the pseudonymous Satoshi Nakamoto and operationalized in 2009 [1], pioneered the concept of a decentralized digital currency based on cryptography and distributed consensus. Its core design revolves around the Unspent Transaction Output (UTXO) model and a simplified, stack-based scripting language [2]. These foundational choices have endowed Bitcoin with remarkable stability and security by deliberately restricting computational expressiveness, thus minimizing potential vulnerabilities [3]. However, this very emphasis on simplicity has also constrained Bitcoin's programmability, limiting its application primarily to straightforward monetary transfers rather than more intricate smart contracts or decentralized applications.

To overcome these limitations, Ethereum was introduced in 2015 with a fundamentally different design philosophy [4]. Unlike Bitcoin's security-oriented UTXO model, Ethereum employs an account-based model. It maintains global state through persistent accounts, where each account stores balances and state variables. Ethereum transactions directly update account states and invoke smart contracts. These contracts are Turing-complete and run on the Ethereum Virtual Machine (EVM). This architecture allows

Ethereum to support rich, expressive decentralized applications. However, Ethereum's flexibility introduces additional complexity and enlarges the potential attack surface [5], reducing the efficiency of transaction parallelization.

The fundamental difference between Bitcoin and Ethereum arises from their distinct design goals. Bitcoin prioritizes simplicity, security, and efficient value transfer at the expense of flexibility and programmability. In contrast, Ethereum pursues flexibility and application richness at the expense of simplicity and security. As depicted in Figure 1, these design choices directly impact how the two blockchains manage and update their states.

With the growing demand for blockchain-based applications, the Bitcoin researchers and developers have sought to enhance programmability while maintaining its security advantage. In Bitcoin's early stages, researchers and developers explored its potential applications beyond payments, especially using its transparent ledger for digital notarization services. They attempted to embed data or hashes of data into the Bitcoin ledger using the OP_RETURN operation to extend its programming capabilities [6]. However, this method had several problems. First, the newly generated UTXOs that contain the OP_RETURN operations could not be used or destroyed, leading to continuous growth in the UTXO set, increasing the storage burden on Bitcoin nodes. Second, to incentivize miners to include these transactions in blocks, users had to pay high transaction fees, which severely limited the widespread adoption of this application. Additionally, the programming codes embedded in the OP_RETURN operations could only be executed and verified off-chain, requiring extra trust assumptions in off-chain nodes, thereby weakening the security and integrity of these applications.

*Corresponding author.

liaoguofu2022@email.szu.edu.cn (Guofu Liao); ttwang@szu.edu.cn (Taotao Wang); slong1007@gmail.com (Long Shi); 2453043004@emails.szu.edu.cn (Jiqun Zhang); xiaiyihan2023@email.szu.edu.cn (Yihan Xia); yang.qing@szu.edu.cn (Qing Yang); xxwu.eesissi@szu.edu.cn (Xiaoxiao Wu); zsl@szu.edu.cn (Shengli Zhang)

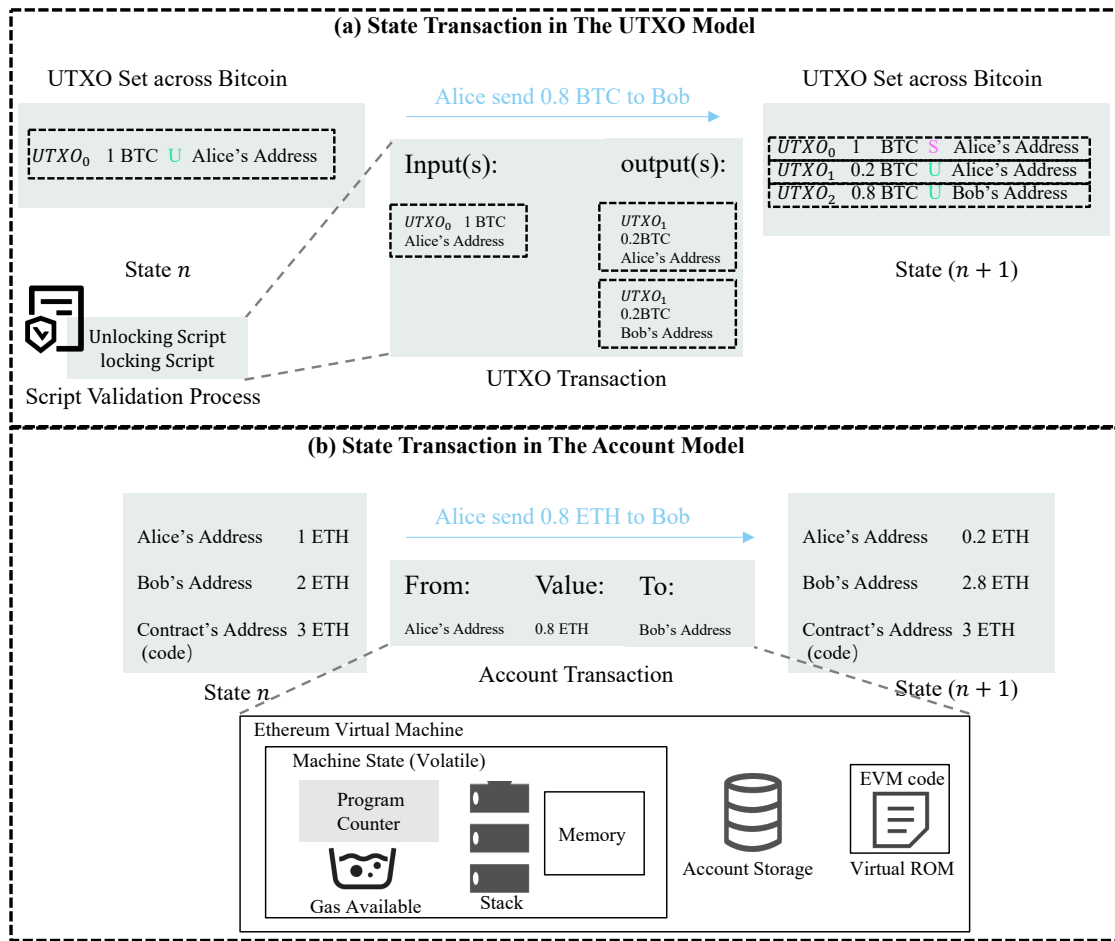


Figure 1: Comparison between Bitcoin's UTXO model and Ethereum's account model.

In 2017, the Bitcoin mainnet experienced a significant technical upgrade known as Segregated Witness (SegWit) [7]. SegWit brought notable improvements by separating signature data (witness data) from the main part of the transaction and moving it to a separate section of the block. Although this change did not alter the physical block size limit (which remains 1MB), it optimized the layout of data within blocks, effectively expanding the amount of transaction data each block could hold to approximately 4MB, thereby increasing the throughput of Bitcoin. More importantly, the introduction of SegWit laid a solid foundation for Bitcoin's scalability. It particularly facilitated the rapid development of off-chain technologies such as the Lightning Network [8]. As a solution to the scalability of the Bitcoin mainnet, the Lightning Network enables real-time Bitcoin transactions off-chain by building a payment channel network between users. Transaction results (such as final balance updates) are submitted to the Bitcoin mainnet for confirmation only when necessary. By optimizing the data structure within blocks, SegWit directly reduces the computational and storage costs associated with creating payment channels, enabling transactions to be processed instantly, and securely closing these channels, making the Lightning Network operate more efficiently. In 2021, the Bitcoin mainnet under-

went another significant upgrade—the upgrade of the Taproot proposal [9, 10, 11]. This proposal introduced the Schnorr signature algorithm and the Pay-to-Taproot (P2TR) transaction type, which not only enhanced Bitcoin's programming capabilities, but also laid the technical possibility for implementing more complex smart contract functions. According to our data, Taproot transactions now account for more than half of all transactions on the Bitcoin mainnet (see Section 9).

The Taproot proposal has led to the emergence of many new protocols such as Ordinals, Atomicals, BitVM, and BEVM, which aim to extend the programming capabilities of Bitcoin. And these new protocols have significantly increased activity on the Bitcoin mainnet, attracting numerous transactions and activities, and directly boosting miner revenues. This positive change has motivated miners to focus more on maintaining the security and stability of the Bitcoin mainnet, gaining widespread support from participants in the community. These new protocols can be categorized into several types: the protocols for implementing NFT functionality on the Bitcoin mainnet, such as the Ordinals Protocol and Atomicals Protocol; the standards for implementing ERC-20-like tokens, such as ARC-20, BRC-20, and Rune; and the smart contract protocols for Turing-complete pro-

programming functions, such as BitVM. Together, these protocols form a new technology and investment track known as the “Bitcoin ecosystem”, attracting broad market interest.

Although the above-mentioned protocols and concepts have garnered widespread attention in the industry, there is still a lack of comprehensive technical survey and academic research on these protocols. This lack of research may lead to delays in knowledge development in the relevant fields and limit the potential for innovation and application. To address this gap, our survey makes the following contributions:

- We detail the technical intricacies of the Taproot upgrade and investigate the Bitcoin Layer 1 protocols that harness Taproot’s distinctive features to program NFT into Bitcoin transactions, including the Ordinals, Atomicals protocols, and the corresponding fungible token standards constructed upon Ordinals and Atomicals (e.g., BRC-20, ARC-20).
- Furthermore, we define some selected Bitcoin ecosystem protocols as Layer 2 solutions akin to Ethereum’s Layer 2, and categorize them into four Layer 2 types. We investigate and compare the effects of these Layer 2 protocols on Bitcoin’s performance and programming capability.
- Finally, by gathering and analyzing data from the Bitcoin blockchain, we obtain the metrics on Bitcoin’s block capacity, miner fees, and the growth trajectory of Taproot transactions initiated by the Bitcoin ecosystem protocols. And the findings from data analysis affirm the beneficial impact of these protocols on Bitcoin’s mainnet.

The structure of the remainder of this paper is as follows. Section 2 reviews related work. Section 3 provides preliminary knowledge of Bitcoin. Section 4 delves into the Taproot proposal. Section 5 examines the Ordinals protocol. Section 6 discusses the Atomicals protocol. Section 7 discusses the differences between ordinals and Atomicals protocol. Section 8 covers Bitcoin Layer 2 protocols. Section 9 introduces our data collection and analysis methods. Section 10 concludes this work and discusses directions for future research.

2. Related Work

In this section, we survey the related literature along three representative lines: (i) research on Bitcoin, focusing on advances in scripting capabilities, cryptographic upgrades, and inscription protocols; (ii) research on Ethereum, which investigates smart contract programmability, security analysis, and scalability solutions; and (iii) research on Solana, which explores architectural innovations for high-throughput blockchain design. In addition, we discuss how our work extends and differentiates itself from each of these research directions.

2.1. Bitcoin-focused Research

Early work by Lande and Zunino [12] developed formal models using dedicated process calculus for precise specification and automatic verification of Bitcoin smart contracts, enabling the identification and categorization of subtle security vulnerabilities and open research challenges. Their work established important foundations for the field, although its scope was shaped by the limited scripting capabilities of Bitcoin at the time.

Following the Taproot upgrade, Jain et al. [13] presents a unified study detailing the cryptographic foundations of Bitcoin through ECDSA and Schnorr signatures, systematically examines the Taproot upgrade’s advancements in privacy and scripting capabilities, and provides in-depth analysis of how Taproot and Schnorr signatures facilitate the optimization of complex multi-party contracts. However, the study remains focused on technical and cryptographic aspects, offering limited discussion of the diverse protocols, ecosystem innovations, and real-world applications that have emerged in the wake of the Taproot upgrade.

Recent studies on Bitcoin inscriptions and Ordinals [14] explored the foundational aspects of Bitcoin-based NFT technologies, providing historical analysis of inscription methods. However, this study’s exclusive focus on the Ordinals protocol neglects other significant innovations, such as the Atomicals protocol, that now shape Bitcoin’s inscription and tokenization landscape.

Despite these contributions, current research lacks systematic coverage of both the foundational technologies extending Bitcoin’s programming capabilities and the diverse ecosystem protocols built upon them. This survey paper aims to fill this gap: we not only cover the detailed descriptions and analyses of Taproot upgrades and Ordinals (as in [13, 14]), but we also more comprehensively explain the basic elements of other relevant technologies and protocols and provide comparative analysis.

2.2. Ethereum-focused Research

Vacca et al. [15] conducted a systematic review of blockchain and smart contract development practices, with a primary focus on Ethereum. Their work surveyed development techniques, analysis tools, and formal verification methods, identifying open research challenges and limitations of current toolchains within the Ethereum ecosystem.

Wang et al. [16] provided an in-depth survey of security enhancement technologies for Ethereum smart contracts, systematically examining vulnerability detection, prevention, and mitigation approaches. Their analysis highlighted both the progress and persistent challenges facing the security of Ethereum-based applications.

Neiheiser et al. [17] systematically reviewed Ethereum Layer 2 scaling techniques, including Rollups, Plasma, and State Channels, and compared their respective trade-offs in scalability, security, and decentralization. They also outlined future directions for improving Ethereum’s throughput.

Di Angelo and Salzer [18] surveyed a broad range of analysis tools for Ethereum smart contracts, comparing

static and dynamic analysis methods and assessing their effectiveness, while identifying limitations and opportunities for tool improvement.

While Ethereum research has extensively explored general-purpose computation and the challenges of Turing-complete smart contracts within an account-based model, our work instead investigates the evolution of programmability on Bitcoin's UTXO-based architecture. Specifically, we focus on how Bitcoin leverages cryptographic advances like Taproot to achieve new levels of programmability without compromising its foundational security properties.

2.3. Solana-focused Research

Wu et al. [19] performed a systematic analysis of Solana's architecture, focusing on its Sealevel parallel execution model, Proof-of-History, Tower BFT consensus, and runtime scheduling. Their study compared Solana's scalability and performance characteristics with those of Ethereum, offering insights into the trade-offs of high-throughput blockchain design.

Kong et al. [20] conducted empirical analyses comparing Solana and Ethereum, evaluating transaction throughput, latency, and fees under varied workloads. Their results provided detailed insights into the impact of Solana's parallel execution (Sealevel) versus Ethereum's serial execution (EVM) on performance metrics.

While Solana research primarily investigates high-throughput blockchain design through architectural innovations such as parallel execution and advanced consensus mechanisms, these studies are rooted in an account-based paradigm. In contrast, our work focus on how Bitcoin maintains its UTXO model and security properties, while enabling new functionalities through the Taproot proposal and further expanding its capabilities via inscription protocols and Layer 2 solutions such as BitVM, Babylon, and BEVM.

3. Preliminaries

3.1. UTXO Model

The addresses on the Bitcoin blockchain are computed from public keys. Each UTXO is associated with a specific owner's address and can only be used by providing a valid signature signed with the private key corresponding to the public key that generates the address. Each Bitcoin transaction consumes a number of UTXOs in its input and creates at least one new UTXOs in its output [21, 2, 22]. Although each UTXO can contain any arbitrary BTC, once created it is indivisible just like a coin that cannot be cut in half. If a UTXO is larger than the desired value of spending transaction, it still must be consumed in its entirety and must generate the change as a separate UTXO in the transaction. In the example depicted in Figure 1(a), if Alice's address is associated with a UTXO containing 1 BTC, which is created by a previous transaction as its output and now Alice wants to pay Bob's address 0.8 BTC, Alice will construct a transaction that takes the UTXO as input and consume the entire 1 BTC and produces two new UTXOs as output: one UTXO



Figure 2: The illustration of Bitcoin's transaction format.

pays Bob's address 0.8 BTC and the other UTXO returns 0.2 BTC as the change to Alice's own address. As a result, most Bitcoin transactions will generate change. Here, we ignore the transaction fees required to pay to miners for packing transactions onto the blockchain.

3.2. Transaction Format

Bitcoin's blockchain is a series of concatenated blocks, and each of block contains a set of transactions [23]. The function of transactions is transferring BTC from a source of funds, called an input, to a destination, called an output. The format of Bitcoin transaction's data structure is encoded into three parts: Metadata, Input, and Output, as illustrated in Figure 2. The metadata part of a transaction includes the following information: the hash of the transaction, the version number of the Bitcoin software, the number of input UTXOs, the number of output UTXOs, the lock time and the memory size occupied by the transaction. The hash value is the unique identifier of each transaction, which is generated by computing the transaction content with the SHA-256 algorithm. The input part references UTXOs that are outputs from other transactions by specifying the hashes of the transaction and the sequence numbers of the outputs. To spend UTXOs, a transaction's input also includes unlocking scripts that satisfy the spending conditions set by the referenced UTXO. For example, the unlocking script is usually a signature proving the ownership of the Bitcoin address specifies in the locking script of the referenced UTXO. The output part creates UTXOs containing an amount of BTC, a Bitcoin address as the recipient, and the locking script that defines the condition for spending the UTXO. In most cases, the locking script will lock the output to a specific address, thereby transferring ownership of the contained BTC to the address of a new owner.

3.3. Script Language

The software of Bitcoin clients verifies the validity of a transaction by executing the scripts contained in the transaction, which are written in a Forth-like scripting language. Script is distinguished by its ease of use, its succinctness, and its stack-based execution engine, however, it is Turing incomplete. With fewer than 200 instructions that are known as opcodes, the scripting language of Bitcoin is designed to support many cryptographic functions [24]. Bitcoin nodes execute these script instructions to perform operations such as hashing and digital signature verification in a stack execution engine. The transaction verification within Bitcoin relies on two types of scripts:

- **Locking Script:** A locking script is written into a transaction output to specifies the conditions that must be satisfied to spend the output in the future. The locking script in the script program is denoted by “script-PubKey”.
- **Unlocking Script:** An unlocking script is written into a transaction input to satisfy the conditions imposed by the locking script of the output UTXO referenced in this input. Most of the time, unlocking scripts contain a digital signature produced by the private key corresponding to the address in the locking script. The unlocking script in the script program is denoted by “scriptSig”.

When the software of Bitcoin clients verifies the validity of a transaction, the unlocking script in each input is executed along with the locking script in the output UTXO referenced by this input to see if it satisfies the spending condition. If the software of Bitcoin clients executes the scripts completely without error returning an error on top of the stack, the transaction is considered valid. The types of transaction outputs can be classified according to the spending conditions specified by its locking script. Important types of transaction outputs include Pay-to-PublicKeyHash (P2PKH), Pay-to-ScriptHash (P2SH), Data Recording Output (OP_RETURN), Pay-to-Witness-PublicKey (P2WPK), and Pay-to-Taproot (P2TR). In the following, we will discuss about these types of transaction outputs with a particular focus on P2TR, which is extensively used to implement Bitcoin Layer 1 and Layer 2 protocols.

3.3.1. Pay-to-PublicKeyHash (P2PKH)

In Bitcoin, a Pay-to-PublicKeyHash (P2PKH) output of a transaction contains the hash of a public key <Public Key Hash> in its locking script to as the recipient of BTC in this output. To spend a P2PKH type of transaction output, it is required to supply a public key <Public Key> and a digital signature <Sig> as the unlocking script in the input that spends BTC in the P2PKH output. The two scripts (the unlocking script and the locking script) together would form the combined validation script for the P2PKH transaction output shown, as shown in Figure 3. The stack of Bitcoin clients will execute the combined validation script without

Unlocking script	<Sig> <Public Key>
Locking script	OP_DUP OP_HASH160 < Public Key Hash> OP_EQUALVERIFY OP_CHECKSIG

Figure 3: The validation script for a P2PKH transaction output.

errors if and only if the following two conditions are satisfied: 1) the public key <Public Key> in the unlocking script can match the hash of public key <Public Key Hash> in the locking script; 2) the signature <Sig> is generated from the private key corresponding to the public key <Public Key> (and thus <Sig> can be verified with <Public Key Hash>).

3.3.2. Pay-to-Script-Hash (P2SH)

The Pay-to-Script-Hash (P2SH) transaction output was developed to simplify the use of more complex and functional scripts as easy as a payment script (i.e., the P2PKH script) that requires the supply of the corresponding signature to unlock. The scripts that implements complex and functional operations beyond simply paying BTC to an address (i.e., a public key hash) is called redeem scripts. An example of such redeem script is a payment script that requires the supply of multiple signatures to unlock. To enable the easy use of such redeem scripts, a P2SH output uses the hash of redeem script <Redeem Script Hash>, it essentially shifts the computational overhead from the transaction that contains this P2SH output to the transaction that references this P2SH output in its input.

We consider an example of redeem script that requires the supply of at least 2 signatures corresponding to the specified 5 public keys, PubKey1, ..., PubKey5, to unlock the payment, as illustrated in Figure 4. We compare the combined validation scripts for this redeem script with and without using P2SH script. Without using P2SH script, the locking script contains the redeem script “2 PubKey1 ... PubKey5 5 OP_CHECKMULTISIG”, indicating that two signatures are required from the five public keys to satisfy the unlock condition, and the unlocking script contains the two supplied signatures <Sig1> <Sig2>. With P2SH script, the locking script contains the hash of the redeem script <Redeem Script hash> and two other opcodes for verifying that the hash of the redeem script is correct when unlocking; the unlocking script contains the two supplied signatures <Sig1> <Sig2> and also the Redeem Script.

Other than encapsulating redeem scripts (which are relatively large) into outputs of transactions, P2SH scripts encapsulate redeem script hashes (which are relatively small) into outputs of transactions, and thus shift the storage burden from the one who creates the output to the one who spends the output. Therefore, with P2SH scripts, the spaces used to store the blockchain and the UTXO set can be saved. To spend the P2SH transaction output, the unlocking script need to contain the redeem script and the signatures that satisfies

Complex Script with P2SH		Complex Script without P2SH	
Redeem Script	2 PubKey1... PubKey5 5 OP_CHECKMULTISIG		
Unlocking script	<Sig1> <Sig2> <Redeem Script>	Unlocking script	<Sig1> <Sig2>
Locking script	OP_HASH160 <Redeem Script hash> OP_EQUAL	Locking script	2 PubKey1... PubKey5 5 OP_CHECKMULTISIG

Figure 4: The combined validation scripts using P2SH scripts and non-P2SH scripts in a multisignature scheme.

Locking script	OP_RETURN < data >
-----------------------	-----------------------

Figure 5: The locking script formed by OP_RETURN.

the unlocking conditions. The stack of Bitcoin client software will execute the combined validation script without errors if and only if the redeem script <Redeem Script> in the unlocking script can match the hash of redeem script <Redeem Script Hash> in the locking script.

3.3.3. Data Recording Output (OP_RETURN)

The Bitcoin Improvement Proposal 70 (BIP70) introduces a operation code that allows non-payment data to be included in transaction outputs through the use of lock scripts starting with the OP_RETURN opcode. The OP_RETURN operation code enables the inclusion of data unrelated to payment in the transaction output, thereby achieving the data recording function. The locking script using OP_RETURN is illustrated in Figure 5, the distinguishing feature of the OP_RETURN transaction output is that it does not need any associated unlocking script to unlock it, making it essentially an unspendable output. This feature ensures that the data recorded in the transaction output cannot be spent or modified by future transactions. Therefore, the OP_RETURN transaction output will be excluded from the Unspent Transaction Output (UTXO) set of full node, to save the space RAM of Bitcoin full nodes.

3.4. Digital Signature

Digital signatures are the basic cryptographic building blocks of Bitcoin and used to verify the validity of transactions. When a user submits a transaction to the Bitcoin mainnet, the digital signature is used to satisfy the following three key functionalities:

- **Authenticity:** The digital signature verifies the identity of the user and the source of the funds contained in a transaction to ensure that only the owner of the corresponding private key can authorize the spending of the associated funds.
- **Integrity:** The digital signature is generated with respect to the transaction data so that any changes on the transaction data after it has been signed would invalidate the signature.

- **Non-repudiation:** Once the digital signature is generated and it is together with the transaction recorded on the blockchain, it becomes a permanent and immutable record to ensure that the initiator cannot later refute the transaction.

The digital signature schemes employed by Bitcoin are the Elliptic Curve Digital Signature Algorithm (ECDSA) [25] and Schnorr Signature Algorithm [10], which both utilize the principles of Elliptic Curve Cryptography (ECC). The Elliptic Curve Discrete Logarithm Problem (ECDLP) is a mathematical puzzle in ECC. The difficulty of ECDLP makes it very difficult to derive a private key from a public key, thus providing strong security guarantee for digital signatures.

ECDSA was used in the original implementation of Bitcoin clients. Later, Schnorr signature is suggested by BIP 340 and incorporated into the implementation of Bitcoin clients after Taproot upgrade. Utilizing Schnorr signatures in Bitcoin offers enhanced transaction efficiency, scalability, and privacy protection in comparison to ECDSA signatures. The advantages of Schnorr signatures include the following. First, Schnorr signatures allow multiple signer to cooperate to generate an aggregate signature, which can reduce the amount of signature data in a transaction when multisignature is required to unlock UTXO, and thus lower transaction fee for the transaction. Second, Schnorr signatures provide better privacy protection by aggregating multiple public keys into a single aggregated public key, hiding the each individual public keys and numbers of multisignatures in transactions. This privacy enhancement makes it difficult to distinguish multisignature transactions from ordinary single-signature transactions. In the following, we will introduce the Schnorr signature algorithm and its aggregate signature algorithm, respectively.

3.4.1. Schnorr Signatures

The Schnorr signature algorithm, introduced by Claus-Peter Schnorr, is known for its simplicity, efficiency, and security [26, 13, 27]. It is widely used in various cryptographic protocols due to its compact nature and robust security. The parameters used in the Schnorr signature algorithm are defined below:

- **Generator Point G** is a predefined point on the elliptic curve secp256k1. The generator point G is a standard

point chosen on the elliptic curve, which serves as a common reference for all cryptographic operations in the Schnorr signature algorithm.

- Schnorr Private Key sk is an integer selected randomly from the finite field defined by the elliptic curve secp256k1. This key is kept secret and is used for signing messages.
- Schnorr Public Key P is a point on the elliptic curve secp256k1, computed as $P = sk * G$. This key is derived from the private key and the generator point and is used for verifying signatures.

The Schnorr signature algorithm consists of three algorithmic components:

$(sk, P) \leftarrow KEYGEN(G)$ is the key generation algorithm that generates the Schnorr private key sk and the Schnorr public key P using a generator point G .

$(s, R) \leftarrow SIGN(sk, m)$ is the signature generation algorithm that produces a signature (s, R) for a given message m using the private key sk and the public key P .

$1/0 \leftarrow VERIFY(P, m, (s, R))$ is the signature verification algorithm that outputs a decision to accept or reject the signature (s, R) using the public key P , the message m , and the signature (s, R) .

Algorithm 1 Schnorr-Key Generation

Input: G (Generator Point)

Output: sk (Schnorr Private Key), P (Schnorr Public Key)

- 1: Randomly select a private key sk from the field defined by the elliptic curve secp256k1.
 - 2: Compute the public key P as $P = sk * G$.
-

Algorithm 2 Schnorr-Signature

Input: sk (Schnorr Private Key), m (Message)

Output: signature (s, R)

- 1: Randomly select an integer r from the field defined by the elliptic curve secp256k1.
 - 2: Compute R as $R = r * G$.
 - 3: Compute e as $e = \text{hash}(R || P || m)$.
 - 4: Compute the signature component s as $s = r + e * sk$.
 - 5: Return the signature (s, R) , where R introduces randomness and uniqueness to each signature.
-

Algorithm 3 Schnorr Verification

Input: P (Schnorr Public Key), m (Message), signature (s, R)

Output: Boolean (True/False)

- 1: Compute e as $e = \text{hash}(R || P || m)$.
 - 2: Verify that $s * G = R + e * P$.
 - 3: If the equation holds, return True; otherwise, return False.
-

The private key sk and the public key P generated by the *KEYGEN* algorithm are foundational parameters for

the Schnorr signature scheme. The *SIGN* algorithm is executed by the signer, and the *VERIFY* algorithm is executed by the verifier. The three algorithmic components are detailed in Algorithms 1-3, respectively. Next, we introduce a particularly useful feature of Schnorr signature, i.e., the Schnorr aggregate signature. Schnorr signatures' mathematical foundation supports a significant advantage over ECDSA: linearity. This property allows Schnorr signatures to aggregate multiple signatures into a single signature, providing significant efficiency and flexibility benefits when developing cryptographic protocols. The parameters used in the Schnorr signature aggregate algorithm are listed as follows:

- n : The number of participants involved in the aggregation.
- sk_i : The private keys of the participants, each selected randomly from the finite field defined by the elliptic curve secp256k1.
- P_i : The public keys of the participants, each computed as $P_i = sk_i * G$, where G is the generator point.
- P_c : The Schnorr aggregate public key, which is the sum of the individual public keys P_i .
- s_i : The individual signature computed by each participant using their private key.
- S : The aggregate signature, which is the sum of the individual signature components s_i .

The Schnorr signature aggregate algorithm consists of three algorithmic components:

1. $(P_c) \leftarrow KEYAGG(n, sk_i, p_i, G)$ is the public key aggregation algorithm that generates the Schnorr aggregate public key P_c using the private keys sk_i , the public key p_i , and the generator point G .
2. $(S, R) \leftarrow SIGNAGG(P_i, sk_i, m, G)$ is the signature aggregation algorithm that produces an aggregate signature (S, R) for a given message m using the private keys sk_i and the public keys P_i .
3. $1/0 \leftarrow VERIFYAGG(P_c, m, (S, R))$ is the aggregate signature verification algorithm that outputs a decision to accept or reject the aggregate signature (S, R) using the aggregate public key P_c , the message m , and the aggregate signature (S, R) .

The three algorithmic components are detailed in Algorithms 4-6, respectively.

3.5. Merkelized Abstract Syntax Trees

BIP114 introduces Merkelized Abstract Syntax Trees (MAST), which extends Bitcoin scripts' functionality by enabling more complex and flexible spending conditions [7, 28]. MAST combines the properties of Merkle trees and Abstract Syntax Trees (ASTs), allowing for the encapsulation and protection of Bitcoin scripts in a compressed form

Algorithm 4 Schnorr Public Key Aggregation

Input: n (Number of participants), sk_i (Private keys), P_i (Public keys), G (Generator Point)

Output: P_c (Schnorr Aggregate Public Key)

- 1: For each participant i :
 - Randomly select a private key sk_i from the field defined by the elliptic curve secp256k1.
 - Compute the public key P_i as $P_i = sk_i * G$.
- 2: Compute the aggregate public key P_c as $P_c = P_1 + P_2 + \dots + P_n$.

Algorithm 5 Schnorr Signature Aggregation

Input: P_i (Schnorr Public keys), sk_i (Private keys), m (Message), G (Generator Point)

Output: Aggregate signature (S, R)

- 1: Select a random number r .
- 2: Compute $R = r * G$.
- 3: Compute the aggregate hash e as $e = \text{hash}(m || R)$.
- 4: For each participant i :
 - Compute their portion of the signature s_i as $s_i = k_i + e * sk_i$.
- 5: Compute the aggregate signature S as $S = s_1 + s_2 + \dots + s_n$.
- 6: Return the signature (S, R) .

Algorithm 6 Schnorr Verification

Input: P_c (Schnorr Aggregate Public Key), m (Message), signature (S, R)

Output: Boolean (*True/False*)

- 1: Compute e as $e = \text{hash}(R || m)$.
- 2: Compute U as $U = e * P_c$.
- 3: Compute V as $V = S * G - U$.
- 4: Verify that V equals R . If true, the signature is valid; otherwise, it is invalid.

[7]. The proposal aims to accommodate complex spending conditions and improves privacy by hiding scripts into the leaves of MAST unexecuted branches.

As depicted in Figure 6, the idea of MAST is to use a Merkle tree to encode the spending conditions of a script as branches. Each leaf node of the Merkle tree represents a spending condition, and each branch includes the path from a leaf node to the script root. For example, if a script includes multiple conditions for spending a transaction output, each condition is encapsulated in a separate branch. To spend the output, users reveal and execute only the branch with the spending condition, keeping the other branches obscured.

P2PKH verifies the validity of a transaction by executing the combined script (consisting of the unlocking script and the locking script) to ensure that the top of the stack returns a true at the end of execution. MAST verifies a transaction's validity differently. To spend a transaction output that utilizes MAST, it is required to check the provided branch script

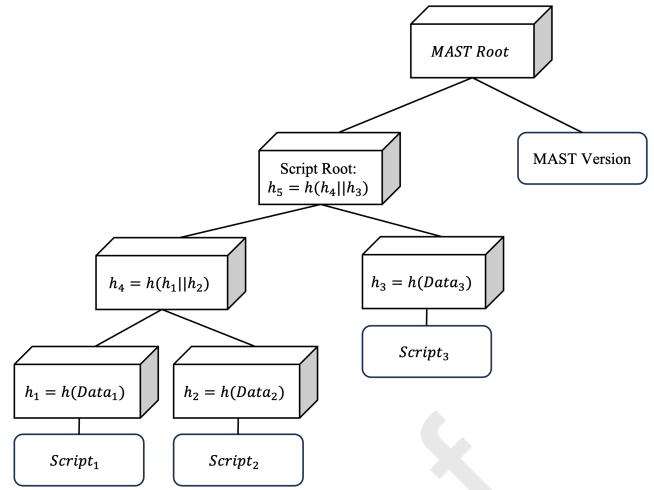


Figure 6: The conceptual illustration of MAST.

and Merkle path against the Merkle root hash to ensure that the stack is empty at the end of script execution.

4. Taproot Proposal

In August 2017, Bitcoin implemented the upgrade of Segregated Witness, which modified the way how transaction data is stored and processed. Segregated Witness improves the transaction storage and processing efficiency by separating witness data (including signature and scripts) from the main transaction data (specifically, from the “script-Sig” field). In Segregated Witness, the witness data, which includes signatures and scripts used to unlock the outputs, is stored offchain separately from the transaction data (such as inputs, outputs, and amounts of BTC) that is recorded on the blockchain. This means that the witness data corresponding to a block does not limit by the 1MB block size. This separation allows more transactions to be included within the limited block size (1MB), and thus improving the block space utilization without increasing the block size.

Taproot is an important upgrade to Bitcoin after Segregated Witness. The corresponding software code was merged into the Bitcoin source core repository by Pieter Wuille in October 2020. After a period of development and promotion, the proposal finally received the necessary miner support on November 14, 2021, with more than 90% of miners reaching a consensus, and was subsequently activated in block 709,632.

The proposal of Taproot contains three Bitcoin Improvement Proposals (BIPs), i.e., BIP340, BIP341 and BIP342. Taproot introduces a new type of transaction outputs called Pay-to-Taproot (P2TR) and the spending conditions specified by its locking script. This helps to minimize the amount of data that needs to be recorded on-chain to reveal the UTXO's spending conditions.

4.1. Generate a Taproot Output Key

In this part, we introduce the locking script of P2TR transaction output. We first introduce how to generate the Taproot output key. In a P2TR transaction output, the

owner's BTC is protected within a 33-Bytes locking script as follows:

<1-Byte witness version>
<32-Bytes Taproot output key Q >

The initial Byte is represented by 0x01, indicating the use of version 1 of Segregated Witness. The subsequently 32 Bytes represents the Taproot output key denoted by Q . The Taproot output key Q is a Schnorr aggregate public key that is computed by aggregating two Schnorr public keys as:

$$Q = P + \text{hash}(P||M) * G$$

where G represents the base point on the used elliptic curve, P represents a point on the elliptic curve, and M represents a Merkle tree root; one of the aggregated two Schnorr public keys $\text{hash}(P||M) * G$, which is derived from the MAST structure; while the other P is a standard Schnorr public key, which is called internal Taproot key.

- The internal Taproot key P is obtained by aggregating one or multiple Schnorr public keys: $P = P_1 + P_2 + \dots + P_n$, where $P_1, P_2, \dots, P_n$ respectively represents n different Schnorr public keys.
- In MAST, the Merkle tree's leaf nodes contain the spending condition specified by the locking script. The Merkle tree root M represents the secure commitment for the spending conditions. It compresses all spending conditions of locking script into a single hash value, thereby users can spend their UTXOs without revealing all spending conditions in the unlocking script.

The Taproot proposal integrates tagged hashing [29] to mitigate collision risks associated with hash functions. Tagged hashes are standard hashes where the tag is concatenated with the data before hashing. Taproot utilizes three distinct tagged hashes:

- $\text{Hash}_{\text{TapLeaf}}$: Used to hash a leaf node.
- $\text{Hash}_{\text{TapBranch}}$: Used to jointly hash sibling nodes within the Merkle tree, representing the conjoint parent node for two scripts.
- $\text{Hash}_{\text{TapTweak}}$: Used to jointly hash the internal public key and the root Merkle tree.

As depicted in Figure 7, the structure of the Taproot output key is generated using the MAST. The procedure begins with creating the leaf nodes that contain the spending conditions specified by the locking script. The $\text{Hash}_{\text{TapLeaf}}$ algorithm is applied to these leaf nodes to produce tags A , B , and C , representing the hashes of the individual spending scripts. Subsequently, the $\text{Hash}_{\text{TapBranch}}$ algorithm hashes pairs of adjacent tags, such as A and B . This process continues recursively, combining pairs of tags until only a single tag remains. In the final phase, a tweak t is generated by hashing the concatenation of the internal public key P and the Merkle root M , using the $\text{Hash}_{\text{TapTweak}}$ algorithm.

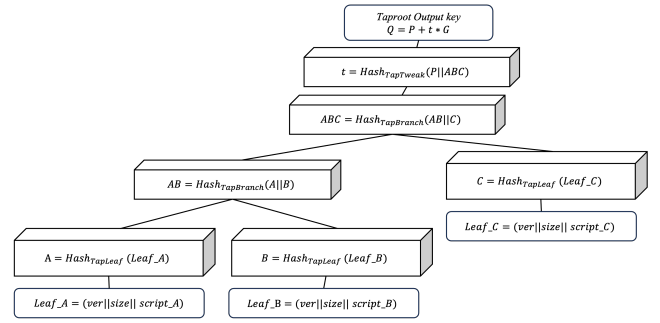


Figure 7: The illustration of MAST that specifies spending conditions.

4.2. Types of P2TR Locking Script

In the locking script of a P2TR transaction output, the Taproot output key Q is derived by aggregating two Schnorr public keys, specifically the internal public key P and another Schnorr public key $\text{hash}(P||M) * G$. In the Schnorr public key $\text{hash}(P||M) * G$, the Merkle root M is computed by recursively combining the hash values of leaf nodes that contain the spending conditions specified by the locking script. If all leaf nodes lack spending conditions, then the Merkle root M is 0. Consequently, based on the value of M , we can categorize the locking scripts of P2TR transaction outputs into two types:

4.2.1. Key Path Spending ($M = 0$)

For this case, the leaf nodes do not contain spending conditions, and the Merkle root M is 0. The calculation formula for the Taproot output key Q is as follows:

$$Q = P + (\text{Hash}_{\text{TapTweak}}(P)) * G$$

Here, the Taproot output key Q is entirely dependent on the internal public key P , meaning that this P2TR transaction output is governed by a single public key.

4.2.2. Script Path Spending ($M \neq 0$)

For this case, the leaf nodes contain one or more spending conditions, and the Merkle root M is computed by recursively combining the hash values of these leaf nodes. When $M \neq 0$, the calculation formula for the Taproot output key Q is as follows:

$$Q = P + (\text{Hash}_{\text{TapTweak}}(P||M)) * G$$

The Taproot output key Q is now determined by both the internal public key P and the Schnorr public key $(\text{Hash}_{\text{TapTweak}}(P||M)) * G$ that is computed through the Merkle root M .

4.3. Taproot for Unlocking Script

In Bitcoin, spending UTXOs requires providing spending conditions in the unlocking script so that Bitcoin client software can validate the transaction. For transaction outputs like P2PKH that are not composed of MAST, the Bitcoin client software validates the transaction output by executing the combined unlocking script and locking script in its stack and then checking if the value returned at the top

of the stack is true after the script execution completes. For transaction outputs composed of MAST, the Bitcoin client software first verifies the transaction by determining if the Merkle root hash, computed from the spending conditions contained in the Merkle tree leaf node and the corresponding Merkle path provided by the unlocking script, matches the actual Merkle root hash. It then checks if the value returned at the top of the stack is empty after the script execution completes. For P2TR transaction outputs, the locking script is divided into key path spending and script path spending scripts, depending on whether the generation of the Taproot output key Q involves MAST (i.e., whether the Merkle root M is 0). This means that there are two formats of unlocking scripts for spending P2TR transaction outputs. Consequently, the Bitcoin client software has two different methods to verify the validity of P2TR transaction outputs. In the following, we discuss the two types of unlocking scripts corresponding to the two types of P2TR transaction output locking scripts.

4.3.1. Unlocking Script for Key path spending

For a P2TR transaction output with a key path spending type locking script, the Taproot output key Q entirely depends on the internal public key P , meaning that the P2TR transaction output is governed by a single public key. When spending this type of P2TR transaction output, the witness data in the unlocking script contains exactly one element, indicating an unlocking script for key path spending. In this case, the element is a signature, and it must be a valid signature for the Taproot output key Q . The full node verifies the transaction's validity by executing the combined unlocking script and locking script, specifically including the execution of the Schnorr aggregate signature verification algorithm to confirm the validity of the transaction's signature.

4.3.2. Unlocking Script for script path spending

For a P2TR transaction output with a script path spending type locking script, the Taproot output key Q is determined by both the internal public key P and the Schnorr public key ($\text{hash}(P||M) * G$) computed from the Merkle root M . When spending this type of P2TR transaction output, if the witness data in the unlocking script contains three elements, it indicates an unlocking script for script path spending. The structure of a UTXO's locking script is as shown in Figure 7. To spend this output, script_A from the MAST leaf node is chosen to form the unlocking script, and this unlocking script is stored in the witness data:

[Scripts]: <Sig>
 $[\text{script}_A]$
 [Control block] (33 + 32n Bytes)

Here, the [Scripts] contains a signature <Sig> for the transaction hash. The $[\text{script}_A]$ corresponds to the tapscript of the Merkle tree leaf node. The [Control block] contains proof information related to script_A . It includes the version number of Segregated Witness, internal public key and the proof path of script_A . In the following, we describe the details of the [Control block]:

[Tapscript Version] (1 Byte): 0xfe & Control block[0]

[Internal Public Key] (32 Bytes)

[Proof Path of script_A] (32 * 2 Bytes): [B], [C]

The Bitcoin client software uses the elements in the witness data to verify the validity of the spent transaction output. This involves determining whether the Merkle root hash calculated from the leaf node and Merkle path provided by the unlocking script matches the actual Merkle root hash.

4.4. Application of Taproot in Multisignature

In P2SH transaction outputs, when spending conditions require multiple signatures, the size of the unlocking script (i.e., witness data) increases with the number of signers. This is because the locking script only contains the redeem script hash <Redeem Script Hash>, while multiple signatures and the redeem script <Redeem Script> are included in the unlocking script. In fact, a P2SH redeem script can contain up to 15 public keys for multisignature verification. This limit is based on the following reasons:

1) The OP_CHECKMULTISIG opcode is designed to verify up to 15 public keys. This design choice balances the efficiency and reliability of script execution with the need to support a reasonable number of signatures in multisignature transactions.

2) Bitcoin has a limit on the total size of each transaction, with a maximum of 1MB. Considering that each public key occupies about 33 Bytes and each signature about 72 Bytes, an unlocking script with fifteen signatures and corresponding public keys approaches this maximum limit.

In contrast, using the Schnorr signature algorithm in P2TR transaction outputs can achieve multisignature schemes with more public keys for verification[30, 31]. The Schnorr signature algorithm leverages its linear property to aggregate multiple participants' public keys $pk_i, i = 1, 2, \dots, n$ into a single public key pk , and aggregate each participant's signature $s_i, i = 1, 2, \dots, n$ into a single signature S . This aggregated public key pk is then used to verify the signature S . This feature allows locking scripts to include more signatures, creating more complex spending conditions. Next, we discuss the n -of- n and m -of- n multisignature schemes using the Schnorr signature algorithm.

4.4.1. n -of- n Signature Scheme

In an n -of- n signature scheme, each participant provides a valid signature to satisfy the spending condition of the transaction output. This means all participants first generate their own Schnorr public key $pk_i, i = 1, 2, \dots, n$, and use the linear combination property of the Schnorr signature algorithm to aggregate these public keys into a single aggregate public key pk , generating a Taproot output key Q as part of the locking script. To satisfy the spending conditions specified by the locking script and construct the unlocking script, all participants generate independent Schnorr signatures s_i on the transaction hash and then aggregate these signatures into a single aggregate signature S as part of the unlocking script. Bitcoin client software executes the combined script (consisting of the unlocking script and locking

script) to verify the aggregate signature. If the top of the stack is true at the end of execution, the transaction output can be spent.

4.4.2. *m-of-n Signature Scheme*

In an *m-of-n* signature scheme, at least m out of n participants (where $m < n$) must provide valid signatures to satisfy the spending conditions of the transaction output. The following discusses the process of implementing the *m-of-n* multisignature scheme in P2TR transaction outputs:

- **Spending Condition Encoding Process:** As depicted in Figure 8, the process of generating the locking script of P2TR transaction output is demonstrated. First, each participant generates their own private key and corresponding public key. These public keys are aggregated into a single aggregate public key, called the internal public key P . Subsequently, the *m-of-n* multisignature scheme (where the spending condition in each leaf node requires at least m out of n participant public keys) is encoded into the Taproot script as leaf nodes of the Merkle tree to generate the Merkle tree root. An additional Schnorr public key $hash_{TapTweak}(P||M) * G$ is computed. Finally, the internal public key P is combined with $hash_{TapTweak}(P||M) * G$ to generate the Taproot output key Q .
- **Spending Process:** To satisfy the spending condition of the transaction output, at least m participants sign the transaction hash using their private keys sk_i . These signatures are aggregated into a single signature S using the Schnorr signature algorithm as the first element of the witness data. Next, the public keys pk_i of the m participants are provided to generate the Taproot script corresponding to the Merkle tree leaf node as the second element of the witness data. The Merkle path from the leaf node to the Merkle root is provided as the third element of the witness data. Finally, the Bitcoin client software uses the provided aggregate signature, leaf node script, and Merkle path proof to verify the validity of the transaction and then decides whether to execute the spend.

5. Ordinals Protocol and BRC-20 Standard

The Taproot upgrade introduced new solutions for implementing more complex programming functions on the Bitcoin blockchain, particularly showing potential in emerging fields such as non-fungible tokens (NFTs) and decentralized finance (DeFi). Based on the technical advantages brought by the Taproot upgrade, Casey Rodarmor launched the Ordinals protocol in January 2023, aiming to enable NFT functionality on Bitcoin [32]. This section will introduce the Ordinals protocol and its derivative BRC-20 token standard.

5.1. Ordinals Protocol

The Ordinals protocol primarily introduces a novel precise numbering system for Bitcoin's smallest units (i.e., "sat" or satoshi), allowing each sat to carry additional metadata (such as text, images, code, etc.), thereby enabling support for non-fungible tokens. This numbering method not only enhances Bitcoin's programmability but also provides potential for developing complex smart contracts and decentralized applications on the Bitcoin mainnet. The core of the Ordinals protocol includes three parts: Number Scheme, Transfer Scheme, and Inscriptions. The following sections will provide a detailed introduction to each part.

5.1.1. Number Scheme

The Bitcoin protocol sets a series of periodic events during its block generation process, such as the Bitcoin halving every 210,000 blocks, the adjustment of block difficulty every 2016 blocks based on the hash rate, etc. The most frequent periodic event is the generation of a new block every 10 minutes. Additionally, here is a difficulty adjustment every two weeks based on the change in hash rate, and the halving event every four years, which significantly impacts Bitcoin's supply and inflation rate. Furthermore, approximately every 24 years (i.e., after six halvings), halving and difficulty adjustment occur simultaneously, forming a conjunction point. The first conjunction point is expected to appear around 2032.

The Ordinals protocol considers that new sats minted in blocks produced at the above periodic events have some rarity. Depending on the block's periodic events and the sequence of new sats within the block, new sats have different rarity levels and can be classified as Common, Uncommon, Rare, Epic, Legendary, and Mythic. For example, the first sat appearing in the genesis block is within the first period and belongs to the first block of both the halving and difficulty adjustment periods, making it of Legendary grade.

The Ordinals protocol provides various methods for representing and classifying sats. Here are examples of different representation methods:

- Integer representation: 1947673583222332, using numerical values to represent a specific sat, corresponding to its mining sequence number.
- Decimal representation: 806277.458222332, using two numbers to represent a specific sat, the first indicating the block height and the second indicating the offset within the block.
- Degree representation: $0^\circ 176277' 1893'' 458222332'''$, with a detailed structure as follows: $A^\circ B'C''D'''$:
 A: Period number, starting from 0.
 B: Block index within the halving period.
 C: Block index within the difficulty adjustment period.
 D: Sat index within the block.

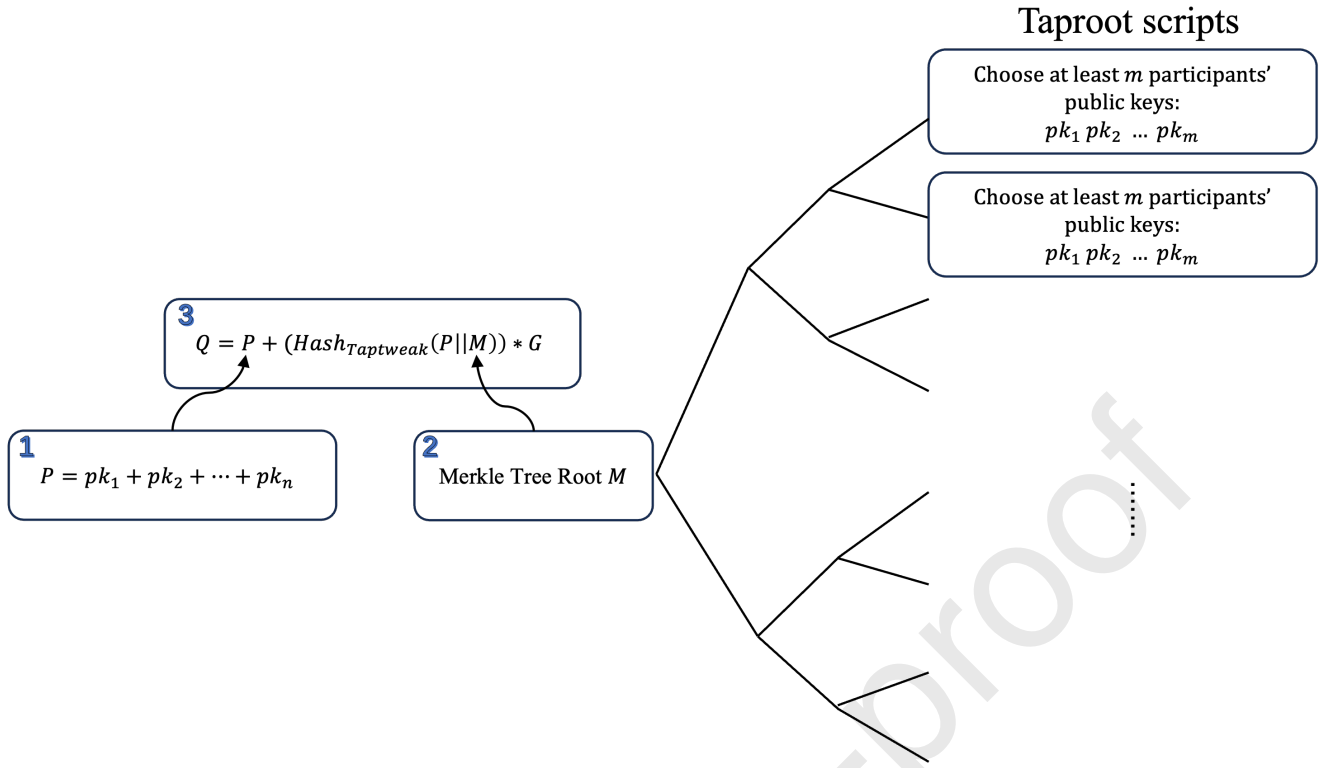


Figure 8: The process of encoding spending conditions with an m -of- n multisignature scheme in a P2TR transaction output.

- Percentile representation: 92.74636120784638%, indicating a sat's position within the overall Bitcoin supply.
- Name representation: abakjcdp11, using alphabetic characters to assign a unique name to the sat.

According to the degree representation method, the rarity classification of sats is as follows: any sat with a non-zero value in any field is considered common, a sat with field D as zero is uncommon, a sat with fields C and D as zero is rare, a sat with fields B and D as zero is epic, and a sat with all fields as zero is mythic. Within the Ordinals protocol framework, the distribution of rare sats is approximately: 2.1 quadrillion common sats, about 6,929,999 uncommon sats, 3,437 rare sats, 32 epic sats, 5 legendary sats, and only one mythic sat.

5.1.2. Transfer Scheme

In the Ordinals protocol, sats are assigned serial numbers based on the sequence in which new BTC is mined, with the first sat in the first block assigned serial number 0, and subsequent sats incrementing sequentially. However, since multiple sats can be included in one transaction output, if it is necessary to accurately identify and transfer specific sats (e.g., those with higher rarity) in a transaction, it becomes complex. To address this issue, Ordinals introduces a transfer scheme to determine how sats are transferred from transaction inputs to transaction outputs. This scheme uses an algorithm to ensure that sats in the transaction input are transferred to the sats in the transaction output in the order of their serial numbers in a first-in, first-out (FIFO) man-

ner, constrained by the size and sequence of the transaction inputs and outputs. This transfer scheme ensures the uniqueness and traceability of each sat, allowing precise management and tracking of specific sats even in complex transactions. It solves the problem of rearranging and reallocating sats during transactions, ensuring reliability and accuracy when using the Ordinals protocol to implement NFTs and other advanced applications on the Bitcoin mainnet.

We use the example in Figure 9 to illustrate the sat transfer scheme in the Ordinals protocol in more detail. Suppose address A initiates a transaction to transfer 20 BTC to address B. In this example, address A, with 50 BTC input, contains sats ranging from [0 to 4,999,999,999]. The transaction transfers 30 BTC back to address A, containing sats ranging from [0 to 2,999,999,999]; the remaining 20 BTC is transferred to address B, containing sats ranging from [3,000,000,000 to 4,999,999,999].

Then, if address A wants to transfer 10 BTC to address C while retaining the 2,599,999,999th sat (as the sat might be very rare), the owner of address A needs to control the sequence of the transaction outputs to ensure that the sat remains in address A's UTXO set, as shown in Figure 9:

- Address A with 30 BTC input contains sats ranging from [0 to 2,999,999,999].
- The first output transfers 10 BTC to address C, containing sats ranging from [0 to 999,999,999].
- The second output transfers the remaining 20 BTC back to address A, containing sats ranging from [1,000,000,000 to 2,999,999,999].

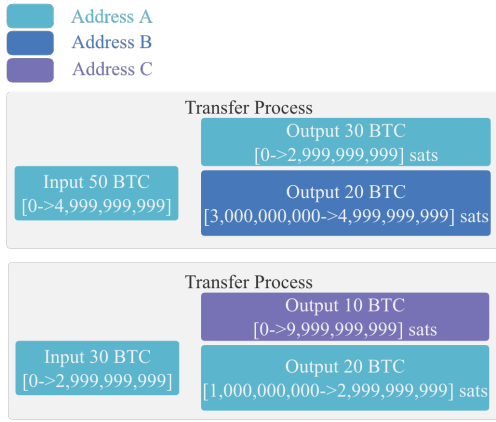


Figure 9: An illustrating example of the sat transfer process in the Ordinals protocol.

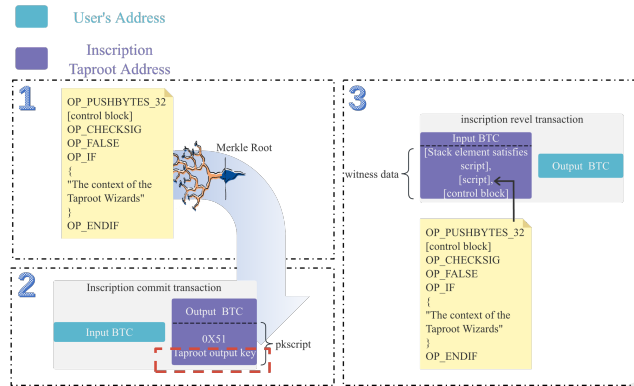


Figure 10: The inscription process of Taproot Wizards.

In this way, the transfer scheme in the Ordinals protocol ensures the management and precise tracking of specific sats, maintaining reliability and accuracy even in complex transaction scenarios.

5.1.3. Inscriptions Process

The Ordinals protocol introduces a mechanism called inscriptions, allowing any content to be “inscribed” onto sats, thereby creating native digital artifacts on Bitcoin, commonly known as NFTs. The inscription process mainly involves two transactions, called the commit transaction and the reveal transaction. First, a Taproot output key is generated in the commit transaction, which is essentially a commit to the inscription content script. After completing the commit transaction, the reveal transaction is executed. This step spends the output of the commit transaction, storing the inscription content on the blockchain for anyone to verify. Figure 10 illustrates the steps of the inscription process using the text string “Taproot Wizards” as the inscription content. Below is a description of the steps shown in the Figure.

In the commit transaction, the locking script of its transaction output contains two fields: [taproot output key] and [0x51]. The [taproot output key] field is a Schnorr aggregate public key, obtained by combining the internal Taproot key P and the hash value of the Merkle root, then multiplying it by the elliptic curve generator G , and finally aggregating with the internal Taproot key P . In the leaf node of

the Merkle tree, the inscription content “Taproot Wizards” is used as the spending condition. The [0x51] field indicates version 1 of Segregated Witness. In the commit transaction, the user submits the inscription content to the Bitcoin blockchain in the form of the Taproot output key, but the content is not yet publicly available on the Bitcoin mainnet. Subsequently, the reveal transaction is initiated, requiring the unlocking script to provide the spending conditions for the UTXO generated in the commit transaction. In the Taproot protocol, the spending conditions are stored in the witness data, containing three elements: [Scripts], [Tapscript], [control block]. The Bitcoin client software uses the elements in the witness data to verify the validity of the spent transaction output. First, the Bitcoin client software computes the Merkle root hash from the leaf node and Merkle path provided by the unlocking script and checks if it matches the actual Merkle root hash. If the root hash matches, it executes stack operations with the [Scripts] from the witness data and the combined script of the locking script. If the top stack element is empty at the end, the reveal transaction is valid. At this point, the inscription content “Taproot Wizards” is publicly available on the Bitcoin mainnet in the form of Tapscript in the unlocking script.

Through the inscription mechanism, text and image content can be inscribed onto specific sats. Once the inscription content is inscribed onto a sat and published on the Bitcoin blockchain, it will be permanently recorded on the blockchain, issuing an NFT that cannot be tampered with or deleted. Transferring the inscribed sat during Bitcoin transactions is equivalent to transferring ownership of the inscribed content, thus achieving the function of NFT ownership transfer.

5.2. BRC-20

The BRC-20 token standard, leveraging the inscription mechanism of the Ordinals protocol, enables the creation of fungible tokens on Bitcoin’s base layer. The name “BRC-20” is inspired by the Ethereum ERC-20 standard. BRC-20 defines a standardized method for deploying, minting, and transfer tokenized assets on the Bitcoin blockchain by inscribing specific JSON-formatted data onto sats. It is important to note that the BRC-20 token standard requires additional third-party ledgers, known as “indexers,” to track users’ token balances. Since BRC-20 is a token standard that cannot implement smart contract functionality, miners on the Bitcoin blockchain will not recognize operations such as the deploy, mint, or transfer of tokens inscribed on sats. Indexers are needed to ensure the new fungible tokens comply with the BRC-20 standard. To illustrate the implementation of BRC-20, this section uses the issuance of a BRC-20 token named “ordi” as an example. Figure 11 demonstrates the operational processes related to “ordi” BRC-20 tokens, describing their operation of deploy, mint, and transfer on Bitcoin.

The operations related to fungible tokens include: Deploy, Mint, and Transfer.

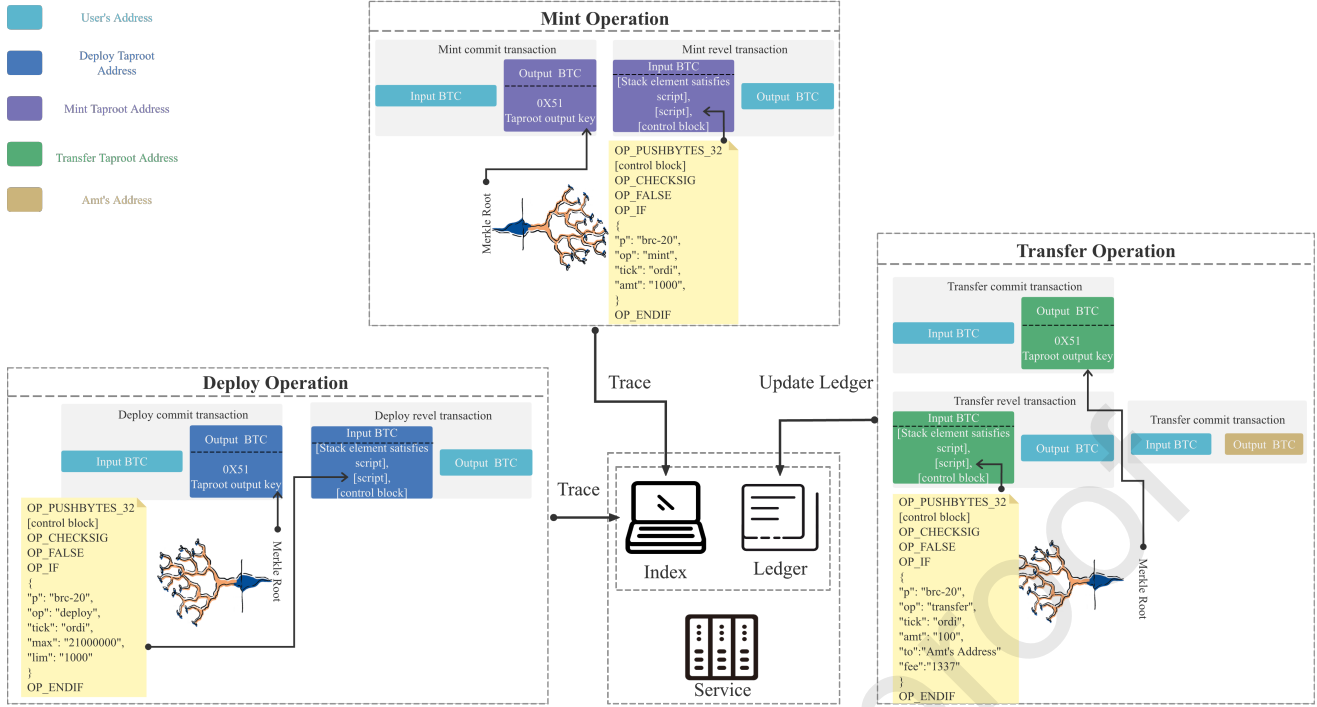


Figure 11: The deploy, mint, and transfer processes of BRC-20.

5.2.1. The Operation of Deploy

For tokens that have not been deployed, users can deploy token information to Bitcoin through commit and reveal transactions. The JSON format of a deploy transaction is as follows:

```
{ "p": "brc-20",
  "op": "deploy",
  "tick": "ordi",
  "max": "21000000",
  "lim": "1000" }
```

The JSON data format of the BRC-20 deploy transaction includes fields such as protocol name ("p"), operation type ("op"), token identifier ("tick"), maximum supply ("max"), and the maximum number of tokens that can be minted at one time ("lim"). In the above JSON data, "brc-20" indicates the protocol used, "deploy" specifies the operation type, "ordi" is the token identifier, the maximum supply is "21000000", and the maximum minting quantity at one time is "1000" tokens.

5.2.2. The Operation of Mint

After the token deployment, any user can mint and trade the tokens. Users write the minting function to the Bitcoin chain through commit and reveal transactions to create and distribute a certain number of BRC-20 tokens. The JSON format of a minting transaction is as follows:

```
{ "p": "brc-20",
  "op": "mint",
  "tick": "ordi",
  "amt": "1000" }
```

The JSON data format of the BRC-20 mint transaction includes fields such as protocol name ("p"), operation type

("op"), token identifier ("tick"), and the number of tokens to be minted ("amt"). In the above JSON data, "brc-20" indicates the protocol used, "mint" specifies the operation type, "ordi" is the token identifier, and "1000" is the number of tokens to be minted.

5.2.3. The Operation of Transfer

Once the total minted tokens equal the total supply, users can only trade or transfer tokens and cannot mint any more. Users write the transfer operation to the Bitcoin blockchain through commit and reveal transactions. Third-party ledgers can recognize transfer operations and obtain transfer addresses and amounts. The JSON format of a transfer transaction is as follows:

```
{ "p": "brc-20",
  "op": "transfer",
  "tick": "ordi",
  "amt": "500",
  "to": "bc1q...",
  "from": "bc1q..." }
```

The JSON data format of the BRC-20 transfer transaction must include fields such as protocol name ("p"), operation type ("op"), token identifier ("tick"), transfer amount ("amt"), recipient address ("to"), and sender address ("from"). "brc-20" indicates the protocol used, "transfer" specifies the operation type, "ordi" is the token identifier, "500" is the number of tokens to be transferred, and both recipient and sender addresses must be Taproot-type addresses starting with "bc1q...".

For BRC-20 token transfers, users first write the transfer operation to the Bitcoin blockchain through commit and reveal transactions. Third-party ledgers can recognize the

transfer operations and obtain the transfer addresses and amounts. If the user's balance in the third-party ledger is greater than the transfer amount, the user can initiate any form of the next transaction to complete the token transfer. To complete an effective transfer of tokens, the specified amount must not exceed the inscribed *overall balance*. The current *available balance* is shown as:

$$\text{Available balance} = \text{Overall balance} - \text{Transferable balance}$$

Where, *Transferable balance* is the total tokens held in the user's wallet. This ensures the executability of token operations within the constraints of the token contract and user token balance.

The BRC-20 token standard brings new functionalities and applications to the Bitcoin ecosystem, opening a new chapter for Bitcoin in the realm of digital assets. However, the market derived from the BRC-20 token standard relies on centralized off-chain indexers, which conflicts with Bitcoin's decentralized nature. Effectively resolving this conflict will be key to the long-term development of the BRC-20 ecosystem.

6. Atomicals Protocol

This section introduces a detailed explanation of the Atomicals protocol, which offers a framework for Non-Fungible Tokens (NFTs) and the ARC-20 token standard [33]. Similar to the Ordinals protocol, the Atomicals protocol standardizes the mint, transfer, and update operations of digital entities on the Bitcoin blockchain. However, unlike the Ordinals protocol, the Atomicals protocol supports self-certification of digital objects and ownership transfer verification. This means digital objects can automatically verify their authenticity and ownership transfer process on the blockchain, eliminating the need for external third parties or indexers. This enhances the integrity of the digital object history within the blockchain ecosystem, making the Atomicals protocol significantly advantageous in maintaining the reliability and security of digital objects.

6.1. Digital Object

The Atomicals protocol provides a framework for minting, transferring, and dynamically updating digital objects (such as non-fungible tokens and fungible tokens) on the Bitcoin blockchain. Compared to the Ordinals protocol, the Atomicals protocol differs in several key aspects.

First, while the Ordinals protocol assigns a unique number to each sat and uses these numbers to identify and track digital objects, the Atomicals protocol assigns a unique Atomicals ID to each digital object. This ID is generated based on the order in which the digital objects are minted, rather than being associated with specific sat.

Second, the minting process of digital objects, although also achieved through two transactions (commit and reveal) similar to the "inscription" method in the Ordinals protocol, involves different data or file formats embedded in the locking script. Additionally, the Atomicals protocol allows for

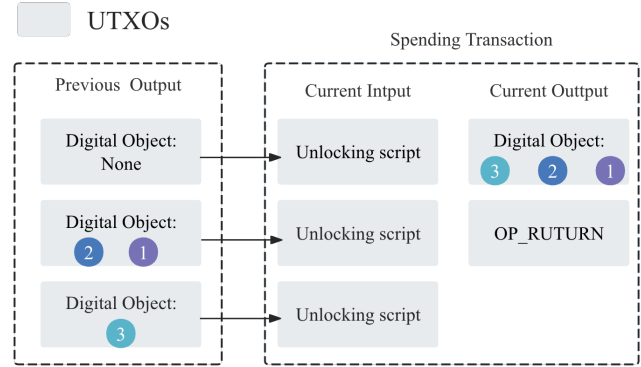


Figure 12: The conceptual illustration of the transfer rule in the Atomicals protocol.

updating the content of digital objects. This means digital objects can be modified or have new information added later. For example, a digital artwork can have new descriptions, tags, or other metadata added after it is minted. To achieve updates, the Atomicals protocol also uses two transactions, commit and reveal.

In terms of transfer rules, the Atomicals protocol is more flexible than the Ordinals protocol. It allows for the use of various address formats, including P2TR, P2SH, and traditional P2PKH addresses. The transfer rules adopt a first-in, first-out (FIFO) method, allocating input digital objects to outputs while ignoring unspendable transaction outputs such as OP_RETURN. During the transfer process, digital objects from the earliest input are allocated to the first suitable output (excluding OP_RETURN), followed by the next digital object being allocated to the next output, and so on. If the input digital objects are insufficient to allocate to all outputs, the remaining digital objects are allocated to the first output. As depicted in Figure 12 illustrates the transfer rules of the Atomicals protocol in detail. In the figure, there are three input digital objects (A1, A2, A3) and two outputs (O1, O2), where O2 is an unspendable transaction output OP_RETURN. According to the FIFO method, A1 is allocated to O1, and since digital objects must ignore unspendable transaction outputs like OP_RETURN, A2 and A3 are also allocated to O1.

It is important to note that when a transaction includes an unspendable transaction output OP_RETURN, the digital objects will skip this output and allocate to the next spendable transaction output. This allocation may cause multiple digital objects to merge into a single unspent transaction output. To split these digital objects, the SPLAT(x) operation can be used. This operation distributes the digital objects across multiple outputs, arranged in alphabetical order based on their unique Atomicals IDs. As depicted in Figure 13 details the SPLAT operation in the Atomicals protocol. The purpose of the SPLAT operation is to distribute multiple digital objects from one output to multiple outputs, ensuring each digital object can exist independently and be accurately tracked. In the figure, we can see that the transaction input contains multiple digital objects that need to be distributed across multiple outputs. The SPLAT operation

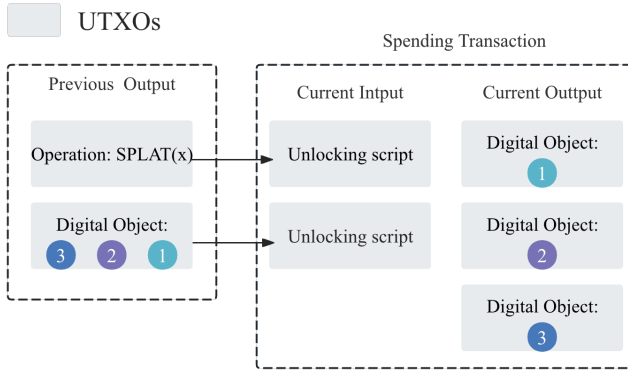


Figure 13: The conceptual illustration of the splat operation in the Atomicals protocol.

allocates them to different outputs by arranging each digital object's unique Atomicals ID in alphabetical order. The inputs contain multiple digital objects to be split, each with a unique Atomicals ID; the outputs are multiple target outputs intended to receive the split digital objects.

In summary, although the Atomicals protocol shares some fundamental design similarities with the Ordinals protocol, such as the two-step commit and reveal scheme, it differs significantly in the identification of digital objects, transfer rules, and content updates. The Ordinals protocol assigns a unique number to each satoshi, whereas the Atomicals protocol assigns a unique ID to each digital object, generated based on the order of minting. In terms of transfer rules, the Atomicals protocol allows the use of various address formats and adopts a FIFO method while ignoring unspendable outputs (such as OP_RETURN). Additionally, the Atomicals protocol supports dynamic updates of digital objects, ensuring the verifiability and immutability of updates through the two-step commit and reveal scheme. These differences make the Atomicals protocol more adaptable and efficient in handling dynamic updates and flexible transfers, making it more suitable for the needs of modern blockchain applications.

6.2. ARC-20

The ARC-20 token standard, developed using the Atomicals protocol as the underlying technology, is a fungible token standard. Similar to BRC-20, the ARC-20 token standard enables the creation of fungible tokens on Bitcoin's base layer. The ARC-20 token standard defines a standardized method for deploy, mint, and transfer tokenized assets on the Bitcoin blockchain. This is achieved by inscribing specific JSON-formatted data onto sats corresponding to the number of tokens. The ARC-20 token standard binds each token to a sat, ensuring that the tokens have a clear value. This design allows ARC-20 tokens to be handled like regular Bitcoin transactions, preserving Bitcoin's divisibility and composability features. Notably, the ARC-20 token standard does not require additional third-party ledgers to track users' token balances. This is because the balance of ARC-20 tokens is directly reflected by the number of sats in the UTXO, eliminating the need for external off-chain indexers to record and manage token balances. Each token's balance

and ownership can be verified and transferred through native transactions on the Bitcoin blockchain, avoiding additional centralized dependencies. Additionally, the ARC-20 protocol ensures the scarcity and minting difficulty of tokens through a PoW system called Bitwork Mining. This means that before minting a token, users need to provide proof of work (PoW) for a field named "Bitworkc". To better illustrate the PoW mechanism in the ARC-20 token standard, we use the example of issuing a fungible token named "Infinity". This is an infinitely minable PoW-based token whose mining difficulty will continue to increase. Currently, the value of the field "Bitworkc" is "888888888" (its decimal value is 586406201480). Before minting, users need to compute a matching string for "888888888" corresponding to a decimal hash value, with a success probability of 1/586406201480. This is because the hexadecimal string "888888888" converts to a decimal value of 586406201480, meaning that among a total of 586.4 billion possible hash value combinations, only one combination can successfully match this string. Users generate hash values by continuously trying different input data (e.g., different random numbers) until they find a hash value that meets the criteria. Assuming a current market GPU (like the RTX 4090) performs about 3 billion SHA256 calculations per second, a user would need approximately 196 seconds to successfully generate a hash value matching "888888888". Once the required hash value is generated, users complete the minting process through commit and reveal transactions. As depicted in Figure 14 demonstrates the operational process related to the "Infinity" ARC-20 token, describing its operation of deploy, mint, and transfer on Bitcoin.

The ARC-20 token standard, similar to the BRC-20 standard, defines three token operations: deploy, mint, and transfer. Just like in the BRC-20 token standard, the ARC-20 standard achieves deploy and mint operations through commit and reveal transactions. In these operations, the commit transaction submits specific JSON-formatted data (containing deployment or minting details) to the Bitcoin blockchain via the Taproot output key, but the content is not yet publicly available on the Bitcoin mainnet. The reveal transaction then discloses the deployment or minting details through the [Tapscript] field in the unlocking script. The detailed process is described in Section 5.

7. Comparative Analysis of Ordinals/BRC-20 and Atomicals/ARC-20 Protocols

In this section, we provide a deeper comparative discussion of emerging Bitcoin protocols, focusing particularly on the differences between Atomicals and Ordinals protocol, as well as between their respective fungible token standards, ARC-20 and BRC-20.

7.1. Protocol-Level Differences

As depicted in Table 1, we compare the core architectural differences between the Ordinals and Atomicals protocols in five key dimensions: indexing and state resolution, address compatibility, collections/grouping, fungible token

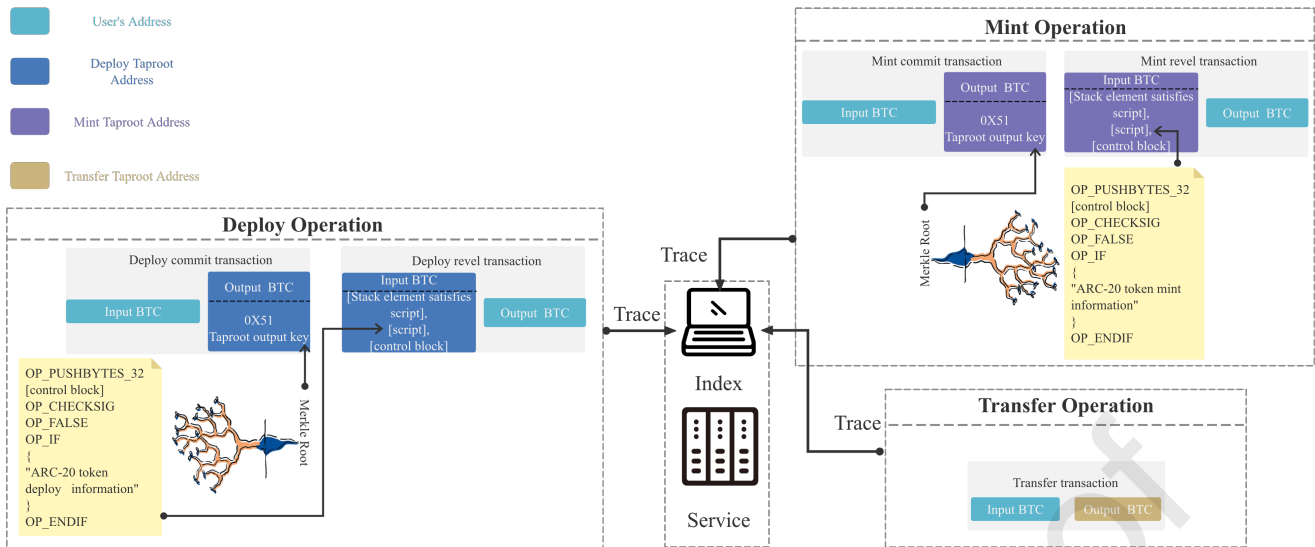


Figure 14: The deploy, mint, and transfer processes of ARC-20.

Table 1
The Comparison of Ordinals and Atomicals Protocols

Feature	Ordinals Protocol	Atomicals Protocol
Indexing and State Resolution	Off-chain indexers define state; no protocol-level enforcement	On-chain state; indexers are auxiliary for querying but not state-defining
Address Compatibility	P2TR only for all operations	Deploy/Mint: P2TR; Transfer: P2TR, P2SH, P2PKH
Collections/Grouping	Not supported on-chain	Native on-chain collections/containers
Fungible Token Support	BRC-20 (It creates a JSON protocol in an Inscription to define the decentralized mint rules)	ARC-20 (It uses Satoshis as unit of account. Decentralized and direct minting modes available.)
Name Service	None (arbitrary, not unique)	Protocol-level, globally unique namespace system

support, and name service. The detailed discussion of these differences is presented as follows.

Indexing and State Resolution: In the Ordinals protocol, asset states (e.g., token balances and ownership) are reconstructed exclusively by off-chain indexers based on inscription metadata, as there is no protocol-level enforcement or validation of state. Consequently, each indexer independently maintains a third ledger for asset states, which can lead to inconsistencies between indexers if their data sources are incomplete or their parsing logic differs.

In contrast, the Atomicals protocol defines asset state deterministically through Bitcoin's on-chain UTXO model. Asset state is enforced by the Bitcoin mainnet itself, and indexers serve only as auxiliary tools for querying existing UTXOs; they do not define or maintain state. Any compliant indexer can independently replay the Bitcoin mainnet to obtain the same verifiable and globally consistent asset state, ensuring trustless and deterministic resolution without reliance on off-chain assumptions.

Address Compatibility: The Ordinals protocol requires all operations to use P2TR addresses, as both its inscription mechanism and asset management are built on Taproot's script path spending, which enables the validation and reve-

lation of inscription data. This design simplifies implementation but limits compatibility with wallets and tools that do not support P2TR, restricting broader ecosystem interoperability.

In contrast, the Atomicals protocol requires P2TR addresses for deploy and mint operations but supports not only P2TR but also other address formats such as P2SH and P2PKH for asset transfers. This flexibility stems from its design, where asset ownership is determined solely by UTXO control rather than Taproot type transaction. As a result, any valid Bitcoin address capable of receiving UTXOs can serve as a destination for asset transfers, ensuring broader compatibility with legacy wallets and infrastructure.

Collections and Grouping: The Ordinals protocol lacks native support for asset grouping or collections; any such organization must be maintained off-chain through marketplace conventions or third-party indexers. This absence of standardized on-chain structures complicates asset management, reduces composability, and increases reliance on external services for collection-level operations.

In contrast, the Atomicals protocol introduces on-chain primitives for containers and collections, enabling structured and composable asset management natively within the protocol. This design enhances the organization, querying, and management of related assets directly on-chain, improving composability and reducing dependence on off-chain infrastructure.

Fungible Token Support: The Ordinals protocol supports fungible tokens through the BRC-20 community standard, which embeds deploy, mint, and transfer instructions via JSON inscriptions. However, asset states are reconstructed off-chain by indexers based on inscription metadata, as the protocol does not maintain on-chain asset state. This reliance on off-chain reconstruction introduces ambiguity, as different indexers may apply inconsistent parsing logic, leading to discrepancies in token balances and ownership records

and ultimately undermining the protocol's trustless guarantees.

In contrast, the Atomicals protocol defines ARC-20 as a native fungible token standard. Each ARC-20 token is uniquely associated with a Digital Object that governs its deployment and minting rules. Token issuance, transfers, and balances are all recorded directly on-chain through UTXO ownership, with token balances determined by the number of satoshis locked in each UTXO. This approach ensures a verifiable and consistent state without reliance on off-chain indexers.

Name Service: The Ordinals protocol lacks any protocol-level naming service or uniqueness enforcement. Asset names, such as token or collection identifiers, are arbitrarily written within JSON inscriptions without any on-chain validation. This approach leads to common risks such as name collisions, duplication, and impersonation. Furthermore, in the absence of protocol-level rules, different off-chain indexers may interpret or resolve asset names inconsistently.

In contrast, the Atomicals protocol implements a hierarchical on-chain namespace, where names are uniquely registered through deterministic, protocol-defined state transitions. Once a name is successfully deployed, any subsequent attempt to register the same name is invalidated during blockchain replay. This mechanism guarantees globally unique, conflict-free identifiers and enables trustless, deterministic name resolution without reliance on off-chain infrastructure.

In summary, the Ordinals protocol pioneered the introduction of inscriptions and NFT assets on Bitcoin, establishing a foundational framework for on-chain digital artifacts. However, as an early design, it exposes fundamental limitations, including the lack of protocol-level enforcement for name uniqueness, standardized asset state management, and deterministic parsing rules. These shortcomings lead to ambiguity, inconsistency and reliance on off-chain indexers, fundamentally undermining trustlessness and consistency across implementations.

The Atomicals protocol was developed to address these gaps by introducing deterministic parsing, a hierarchical on-chain namespace for unique asset identification, and structured asset management through native containers. This design enables trustless and consistent state reconstruction, globally unique naming, and verifiable asset management entirely on-chain, eliminating reliance on off-chain services and ensuring the integrity of digital objects within the Bitcoin ecosystem. However, while Atomicals protocol addresses several of the inherent limitations of Ordinals protocol, it introduces new challenges from its reliance on UTXO-based state modeling. Specifically, Atomicals increases the complexity of on-chain state by introducing hierarchical namespaces, and container structures that require explicit tracking through Bitcoin's UTXO model. This design leads to significant growth in the number of small, fragmented UTXOs, as each token or asset transfer often produces additional outputs. Consequently, transaction structures become

more complex, involving multi-input and multi-output operations to manage scattered UTXOs, which reduces transaction efficiency and increases fees. These trade-offs, while enhancing verifiability and trustless property, impose additional burdens on Bitcoin's scalability and long-term UTXO set management.

7.2. Token Standard Differences: BRC-20 vs. ARC-20

As depicted in Table 2, we compare the key differences between the BRC-20 and ARC-20 token standards in terms of their underlying data models, reliance on off-chain indexers, the use of third-party ledgers, whether token balances are represented via UTXOs, and the number of transactions required for deploy, mint, and transfer operations. Specially, BRC-20 protocol relies entirely on off-chain indexers to reconstruct token balances from inscription metadata. Token ownership and state are not verifiable on-chain, introducing ambiguity and potential inconsistency. In contrast, ARC-20 protocol uses a coloring method for token issuance, where the balance of each token is represented by the number of sats in a UTXO (unspent transaction output). This means each token corresponds to a sat, i.e., 1 token equals 1 sat. Due to this correspondence, the transfer of ARC-20 tokens can be handled like regular Bitcoin transactions. This design allows ARC-20 tokens to maintain a high degree of decentralization and transparency, avoiding reliance on external indexers.

Both BRC-20 and ARC-20 protocol rely on indexers for practical usability; however, their roles differ fundamentally. For BRC-20 protocol, off-chain indexers act as the third authoritative ledgers that define token balances and validate transactions, as the protocol itself lacks any on-chain state enforcement. This reliance introduces trust assumptions and potential inconsistencies across indexers. In contrast, ARC-20 protocol maintains all state on-chain through UTXO-based accounting. Indexers serve only as auxiliary tools for querying existing UTXOs; they do not define state. Any compliant indexer can independently replay the blockchain to verify balances, ensuring deterministic and trustless ownership without reliance on any specific third party.

The transfer of BRC-20 tokens involve multiple transactions, including commit and reveal inscriptions, followed by moving the associated sats. This fragmented process increases transaction count, raises fees, and complicates state tracking. By contrast, the transfer of ARC-20 tokens through a single standard Bitcoin transaction, inheriting Bitcoin's native security and finality while minimizing overhead.

In summary, the core distinction between BRC-20 and ARC-20 protocol lies in their fundamental approach to state and ownership. BRC-20 protocol treats token state as off-chain metadata reconstructed by indexers, introducing trust assumptions, fragmentation, and operational inefficiency. In contrast, ARC-20 protocol defines token state directly through Bitcoin's UTXO model, ensuring that balances are verifiable on-chain without reliance on any third party. This trustless, deterministic design achieves greater

Table 2
Comparison of Different NF Standards

Protocol	Based UTXO	Index	The third Ledger	Number of deploy transactions	Number of mint transactions	Number of transfer transactions
BRC - 20	Not	Yes	Yes	2	2	3
ARC - 20	Yes	Yes	Not	2	2	1

security, transparency, and efficiency, aligning more closely with Bitcoin's native principles.

8. Bitcoin Layer 2 Protocols

The Ordinals and Automicals protocols write programs directly into the transactions that are stored on the Bitcoin blockchain, so they can be clearly classified as the Layer 1 protocols of Bitcoin. While there is a comprehensive definition of Layer 2 for Ethereum [34], Bitcoin lacks a clear definition in this regard. Nevertheless, the Bitcoin community has proposed many other solutions that exploit off-chain mechanisms to extend the programming capability and performance of Bitcoin, even if they have not been formally designated as Layer 2 solutions. Drawing inspiration from Ethereum's Layer 2 definition, we define these off-chain-based protocols in Bitcoin as Layer 2 solutions and categorize them into four distinct types (i.e., Rollup, sidechains, client-side verification, and state channel):

- **Rollup:** It is a technology that shifts complex transactions or computations off the Bitcoin blockchain for processing and then submits proof of these computations (such as zero-knowledge proofs or optimistic proofs) back to the blockchain for validation. By combining off-chain processing with on-chain validation, Rollup effectively addresses the limitations of block time and block size, enhancing the secure processing capabilities of the blockchain.
- **Sidechains:** It solves Bitcoin's scalability problem by creating an independent blockchain that runs parallel to the Bitcoin mainchain. These sidechains can generate blocks more quickly and accommodate larger blocks to handle a higher volume of transactions. Sidechains interact with the Bitcoin blockchain through specific bridging mechanisms, such as multi-signature or Isomorphic Binding, allowing for the cross-chain transfer and validation of assets and data.
- **Client-side validation:** It is a transaction verification framework that shifts part of the transaction verification logic and process from the Bitcoin blockchain to the client for independent execution, while integrating with the final confirmation steps on the Bitcoin mainchain. Under this framework, the client uses its built-in verification engine to thoroughly review transactions, ensuring they comply with protocol rules. Once verified, the client generates proofs of the results (such as validating digital signatures or checking the sufficiency of transaction balances) and submits them

to the Bitcoin blockchain for final confirmation and record-keeping. This method allows users to complete transaction verifications quickly and accurately, even during network congestion, without waiting for block confirmations on the Bitcoin mainchain, thereby addressing Bitcoin's scalability challenges.

- **State channels:** It creates a parallel environment to the mainchain, designed to meet the performance demands of high-frequency, real-time transactions. Represented by the Lightning Network, state channels establish one or more secure off-chain channels that allow participants to complete a large volume of transactions rapidly, only submitting final results to the of Bitcoin blockchain when necessary. This mechanism significantly improves transaction efficiency, particularly in scenarios with frequent, small payments.

In this section, we present an in-depth analysis of Bitcoin Layer 2 protocols, focusing on BitVM as a representation of Rollup technology. We will examine its core principles and implementation. Additionally, we will present several sidechains protocols, including Babylon, BEVM, RGB++, and STACKS, discussing their underlying concepts and technical frameworks. We will also present client-side validation, with the RGB protocol as a key example, to explain its design and technical characteristics. Finally, we will present the state channel concept, using the Lightning Network as a representative to explain its working principles and technical structure.

8.1. BitVM

The BitVM protocol [35] is based on rollup technology to extend the Bitcoin functionality, which introduces the function of Turing-complete virtual machine to Bitcoin. The virtual machine of BitVM enables the execution of smart contracts within the infrastructure of the Bitcoin blockchain. Due to the constraint imposed by the scripting language and stack-based virtual machine of Bitcoin, the on-chain computation ability of Bitcoin is limited and cannot achieve Turing completeness. The virtual machine of BitVM achieves Turing-complete smart contracts by offloading computationally complex computations to off-chain. As a result, this architecture effectively alleviates the computation burden on the Bitcoin blockchain. Moreover, leveraging the sophisticated methodologies such as Optimistic Rollups for fraud proofs and challenge-response protocols (which originally proposed for Ethereum Layer 2) [36], BitVM can maintain security for its off-chain smart contract computations.

There are two distinct roles in the operational model of

BitVM: the prover and the verifier. Together, they are responsible for constructing a program and associating this program to a Taproot address. The computation ability of the constructed program is very broad, encompassing any Turing-complete functions capable of executing complex computational tasks. Both the prover and verifier engage in complex, stateful computations off-chain, without recording the details of these computations on the blockchain. After the prover finishes the execution of the program, the prover needs to make a commitment to the execution process. The verifier then can send a challenge to the prover for the purpose of verifying whether the prover's program is correctly executed as committed by the prover. The prover has to send back response to the challenge by revealing the commitment. If the revealed commitment about the prover's program execution are found to be false, the verifier has the right to sanction the prover. This challenge-response mechanism forces the prover to provide the correct statement.

Under the BitVM protocol, the execution of the program is expressed as a binary circuit. In Figure 15, we give a simple binary circuit as the example. A binary circuit is composed of logic gates that perform AND, OR, and NOT operations. Notably, the logic gate of the NAND (NOT AND) operation can be used as a universal logic gate capable of constructing the logic gates of all other operations [37]. Therefore, we can construct binary circuits using only NAND gates. Meanwhile, within Bitcoin's scripting language, a combination of the opcodes `OP_BOOLAND` and `OP_NOT` can facilitate the creation of a new opcode, "`OP_NAND`", which can perform the NAND operation. For example, the binary circuit given in Figure 15 consists of four NAND gates. Each NAND gate contains two inputs and an output, and the state of each input/output is represented as a bit variable that takes a value of 0 or 1.

Now, to make a commitment to the execution process of the program, the prover can make commitment to the binary circuit, called Binary Circuit Commitment, which is used to ensure the correctness and completeness of the program execution process. A Binary Circuit Commitment consists of several Logic Gate Commitment, and each Logic Gate Commitment corresponds to each logic gate in the binary circuit. For example, the Binary Circuit Commitment of the binary circuit in Figure 15 consists of four Logic Gate Commitments. Moreover, each Logic Gate Commitment contains three Bit Value Commitments, corresponding to the state bit variables of the two inputs and one output of the logic gate. These commitments are public to the verifier, who can use them to verify the correctness of the program execution. In the following, we will explain what Bit Value Commitment, Logic Gate Commitment and Binary Circuit Commitment are, how to use Bitcoin scripts to implement them, and how to use them to implement the challenge-response mechanism of BitVM.

8.1.1. Bit Value Commitment

Bit value commitments are fundamental constructs in the BitVM protocol. It enables a prover to assign a bit variables

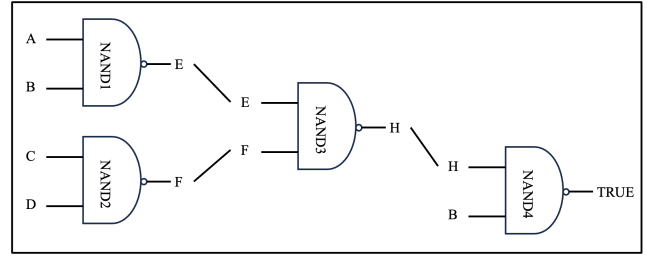


Figure 15: An example of binary circuit to represent the program execution process.

of "0" or "1" to the two inputs and one output of the logic gate. The following Bitcoin script is about to create a UTXO that embodies a Bit Value Commitment.

<code>OP_IF</code>	<code>/* An opcode used to execute conditional branches in Bitcoin scripts. */</code>
<code>OP_HASH160</code>	<code>/* An opcode used to compute the RIPEMD-160 hash of the top stack item. */</code>
<code><HashA1></code>	<code>/* A placeholder for the hash value, representing the HashA1 value. */</code>
<code>OP_EQUALVERIFY</code>	<code>/* An opcode used to compare the top two stack items for equality and verifies the result as true. */</code>
<code><1></code>	
<code>OP_ELSE</code>	<code>/* An opcode to switch to the else branch. */</code>
<code>OP_HASH160</code>	
<code><HashA0></code>	<code>/* A placeholder for the hash value, representing the HashA0 value. */</code>
<code>OP_EQUALVERIFY</code>	
<code><0></code>	
<code>OP_ENDIF</code>	<code>/* The end marker of the conditional statement. */</code>

First, the prover selects two preimages, representing the values "0" or "1" of the state bit variables for the logic gate's input/output. These are denoted as `<PreimageA0>` and `<PreimageA1>`. The corresponding hash values, `<HashA0>` and `<HashA1>`, are computed using the `OP_HASH160` opcode in the Bitcoin script. Then, the prover generates a Bit Value Commitment by creating a locking transaction for a UTXO containing a script that includes `<HashA0>` and `<HashA1>`. This script can be abstracted as "`<HashA0><HashA1>OP_BITCOMMITMENT`".

After that, the prover reveals the preimage corresponding to either `<HashA1>` or `<HashA0>` to the verifier by fulfilling the unlocking conditions of the UTXO containing the Bit Value Commitment. The verifier then verifies the correctness of the prover's Bit Value Commitment by using the script-locked `<HashA0>` or `<HashA1>` along with the revealed preimage to check the integrity of the commitment.

The entire verification process can be performed by cascading the preimages of <HashA0> or <HashA1> with the Bitcoin script used to create the Bit Value Commitment.

8.1.2. Logic Gate Commitment

In the BitVM protocol, a Logic Gate Commitment consists of three Bit Value Commitments corresponding to the two inputs and one output of a logic gate. These Logic Gate Commitments are further interconnected to form a Binary Circuit Commitment. This method transforms the program's execution into a structured format that is both compatible with and executable within Bitcoin Script.

Assume the inputs of the logic gate are A and B, and the output is C. The prover generates a Logic Gate Commitment using the following Bitcoin script:

```

<HashC0><HashC1>OP_BITCOMMITMENT
OP_TOALTSTACK      /* An opcode used to move top
                    stack element from the main stack to
                    the altstack. */
<HashB0><HashB1>OP_BITCOMMITMENT
OP_TOALTSTACK      /* Same as before. */
<HashA0><HashA1>OP_BITCOMMITMENT
OP_FROMALTSTACK    /* An opcode used to move the top
                    stack element from the alternative
                    stack back to the main stack. */
OP_NAND             /* An opcode used to perform a
                    logical NAND operation on the
                    top
                    two elements of the stack. */
OP_FROMALTSTACK    /* Same as before. */
OP_EQUALVERIFY      /* An opcode used to compare the
                    top two elements of stack, and if
                    they are equal, the script
                    continues execution. If not, the
                    script terminates immediately. */

```

In this script, three Bit Value Commitments are combined with an OP_NAND script to generate a Logic Gate Commitment. This script can be summarized as “<HashC0><HashC1><HashB0><HashB1><HashA0><HashA1>OP_GATECOMMITMENT”. Similar to Bit Value Commitments, the prover must reveal the preimages for <HashA0>/<HashA1>, <HashB0>/<HashB1>, and <HashC0>/<HashC1> to the verifier. Finally, the verifier performs an off-chain verification of the Logic Gate Commitment based on the revealed preimages. If the verification process results in a true value at the top of the stack, the logic statement $A \text{ NAND } B == C$ is confirmed.

8.1.3. Binary Circuit Commitment

In the previous section, we defined Logic Gate Commitments and provided an example of generating them using NAND gates. Since any binary circuit can be represented by a combination of NAND gates, the binary circuit of the program execution shown in Figure 15 can be represented by four NAND gates. Consequently, we can use four Logic

Gate Commitments to represent the Binary Circuit Commitment.

In practical applications, the corresponding binary circuit typically consists of many logic gates, potentially numbering in the thousands. To optimize on-chain resource utilization and minimize associated costs, the prover can place the scripts for these Logic Gate Commitments in the leaf nodes of a Taproot tree structure. Then, by performing a series of calculations on the leaf nodes, we can obtain the Taproot output key Q . This method combines the scripts of the Logic Gate Commitments into the Taproot Commitment and stores it within the Taproot output key. This approach leverages the efficiency of the Taproot structure to manage complex operations while maintaining the integrity and cost-effectiveness of on-chain transactions. Figure 16 provides an example of a Taproot Commitment representing a Binary Circuit Commitment.

8.1.4. Pre-signed Off-chain Transactions

In Bitcoin, normal transactions need to be broadcast to the network for confirmation immediately after they are created and signed by the user. However, pre-signed transactions are handled differently from normal transactions. A pre-signed transaction is one that has been signed in advance but not immediately broadcast. It will be included in a new block only when certain conditions (such as time locks) are met.

In the BitVM protocol, the prover and verifier generate these pre-signed transactions using joint signatures to ensure that the script content in the transaction inputs and outputs conforms to the specific format requirements of each step in the protocol. Specifically, in BitVM, a series of challenge or response transactions required when the verifier questions the prover's claims are jointly signed off-chain by both the prover and verifier. These transactions are linked together through the challenge and response mechanism. Joint signatures ensure that each transaction is correctly structured, maintaining the operational integrity of the protocol.

8.1.5. Challenges and Responses

In the BitVM protocol, if the verifier has no doubts about the binary circuit execution committed by the prover, they can execute any agreed-upon operation using a 2-of-2 multisig mechanism. This includes contract settlement, fund transfers, and state updates. For instance, the prover and verifier can jointly sign a transaction to close a payment channel and submit it to the Bitcoin mainnet for confirmation. However, if the verifier questions the prover's binary circuit commitment, the challenge and response mechanism is triggered. In this process, if the verifier discovers inconsistencies in the prover's binary circuit execution claims, the verifier is entitled to seize the prover's collateral. The entire process involves several steps, including jointly initiating the binary circuit setup transaction, the prover revealing preimages, the verifier initiating a challenge, and the prover responding to the challenge, as illustrated in Figure 17. First, both parties jointly initiate the binary circuit setup transaction (TX1:

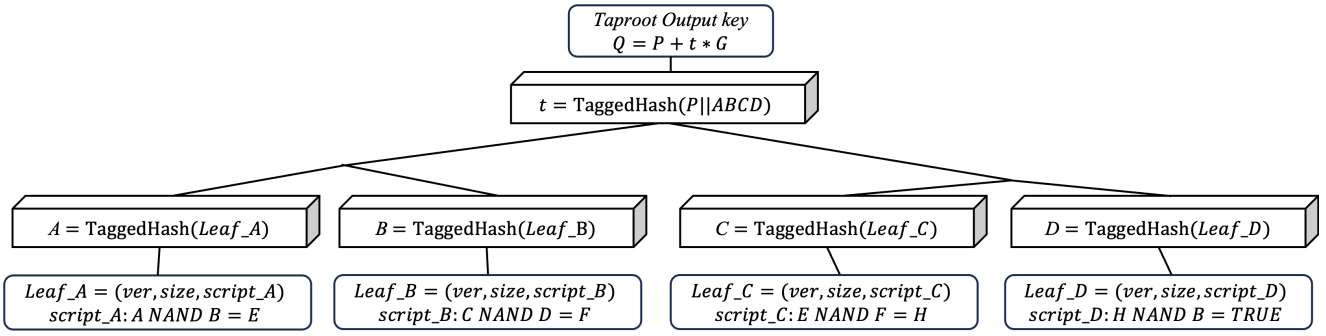


Figure 16: The illustration of the Taproot commitment that represents the Binary Circuit Commitment.

setup). This transaction locks both parties' funds on the mainnet and creates the collateral, laying the foundation for the challenge and response mechanism. This setup specifies that when the verifier challenges the prover, the prover must reveal all preimages involved in the binary circuit to validate the correctness of their claims. Next, the prover sends a reveal transaction (TX2: prover reveal) that reveals all the preimages associated with the binary circuit commitment, including all the bit commitments of inputs and outputs involved in each logic gate.

The verifier then sends a challenge transaction (TX3: Challenge), selecting a specific NAND logic gate to challenge by requesting the preimages of the inputs and outputs of that gate. Finally, the prover responds with a transaction (TX4: Response) that provides the preimages of the challenged NAND gate's inputs and outputs to prove the correctness of their computation. The verifier validates the commitment based on the provided preimages. If the verification succeeds, the process may continue with further challenges and responses; otherwise, the verifier may confiscate the prover's collateral.

This challenge and response cycle repeats until all logic gates are verified or the verifier ceases to challenge the prover. In the following, as depicted in Figure 17, we will detail each step of the challenge-response mechanism and the associated transactions. For simplicity, we omit the typical miner fees associated with each transaction.

Initially, the verifier and prover jointly initiate the Setup transaction. In this transaction, both parties agree to deposit a certain amount of BTC as collateral (in the example shown in Figure 17, each party deposits 1 BTC), and output a Taproot-type locking script. During the challenge and response mechanism, if the prover is found to act maliciously, the verifier can claim all the collateral as a penalty. Conversely, if the verifier does not detect any malicious behavior by the prover during the process, the prover can claim all the funds as a reward. This mechanism ensures that the prover completes the challenge and response process on time, or else they face financial loss.

The Taproot locking script output includes two unlocking conditions. The first unlocking condition requires the prover to provide the preimages of all inputs and outputs of the logic gates in the binary circuit. As shown in Figure 15,

the inputs and outputs of the logic gates in the binary circuit include A, B, C, D, E, F, H, and TRUE. In other words, some leaf nodes in the MAST are the bit commitments for all inputs and outputs in Figure 16. The second unlocking condition is a time lock, meaning that after a certain period, if the prover fails to provide the necessary preimages to spend the UTXO (i.e., does not demonstrate any inaccuracies in the verifier's committed computation), the verifier can unlock the UTXO using only their signature to claim the BTC collateralized in the transaction.

Next, the prover generates a reveal transaction to unlock the locking script created by the setup transaction. To avoid forfeiting the collateralized BTC, the prover must provide the preimages of all logic gates in the binary circuit as part of the unlocking script in the reveal transaction. This satisfies the unlocking condition of the UTXO created by the setup transaction and generates a new UTXO. The locking script for this new UTXO is of Taproot type, with the MAST's leaf nodes consisting of three parts. The first part of the MAST's leaf nodes includes the commitments of all logic gates in the binary circuit (as in Figure 15), placed in different leaf nodes of the MAST. This allows the verifier to select and unlock the preimages of the inputs and outputs of a logic gate provided by the prover. The second part is activated if the verifier detects a contradiction in the prover's revealed content. For example, the prover reveals the output value on D from the challenged NAND1 gate as D equals 1. However, the verifier finds that the value is D equals 0 upon validating the prover's pre-image, constituting a contradiction. In such cases, the verifier can unlock the script by obtaining two different preimages of D and claim all funds. The third part is a time lock. If the verifier does not further challenge the prover's revelation, the prover can transfer the funds with their signature after a certain period.

If the verifier continues to doubt the prover, they can challenge the inputs and outputs of any logic gate in the binary circuit. The verifier assembles the preimages of the inputs and outputs of the selected gate into an unlocking script. This script satisfies the unlocking condition of the UTXO created by the second transaction. Using this unlocking script, the verifier generates a challenge transaction to challenge the commitment of the selected logic gate.

The locking script of the response transaction consists of

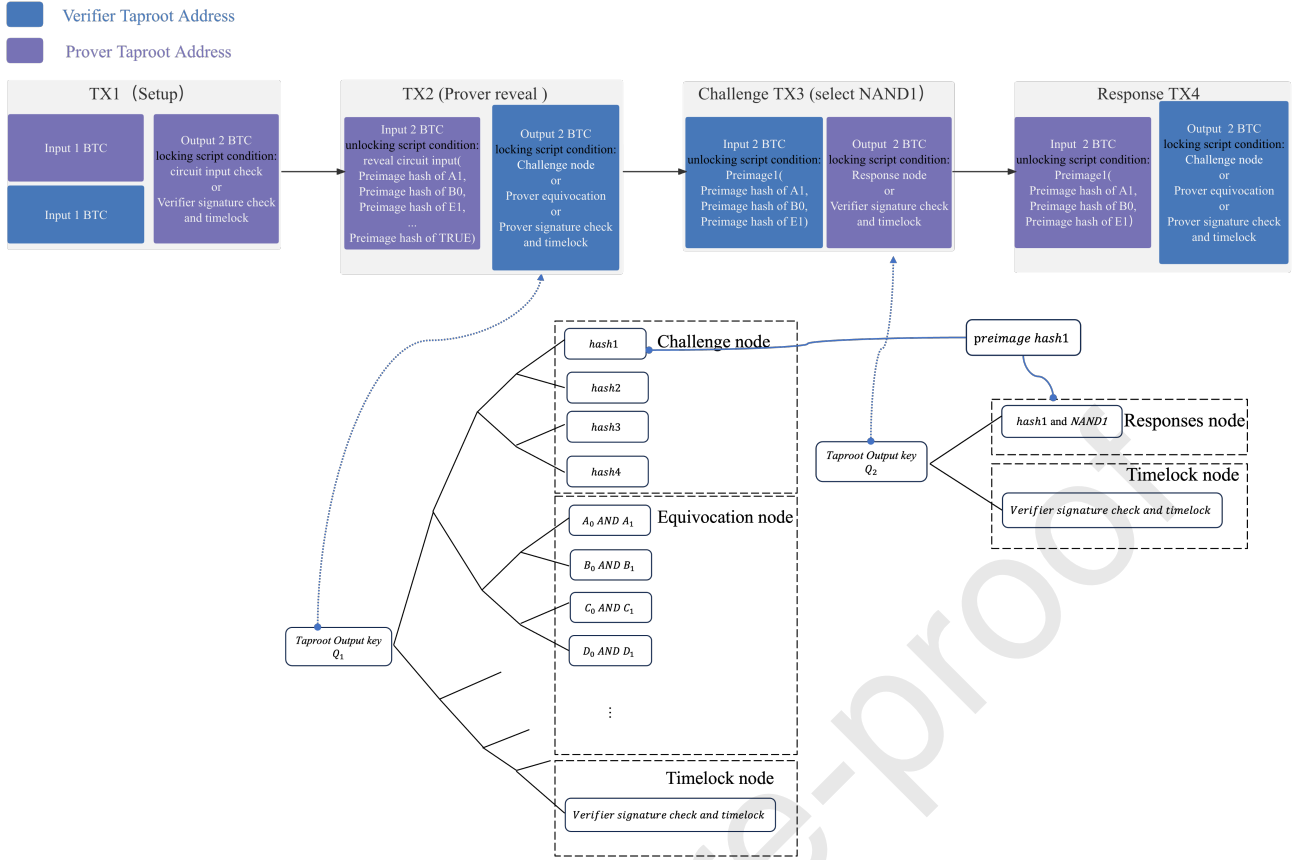


Figure 17: The illustration of the challenge and response process in BitVM.

a MAST. The leaf nodes of the MAST are made up of two parts. The first unlocking condition of the script requires the prover to use the preimages corresponding to the logic gate selected by the verifier to unlock it. The second part of the locking script is a time lock. If the prover does not initiate a response transaction in time, the verifier can transfer all the funds using their signature after a certain period.

Finally, the prover issues a response transaction to address the challenge transaction. In this response transaction, the prover uses the preimages corresponding to the logic gate selected by the verifier as the unlocking script, satisfying the unlocking condition of the UTXO created by the third transaction. The composition of the new locking script's MAST leaf nodes is identical to that in the second transaction. The first part allows the verifier to select and challenge the revealed preimages of the logic gate, the second part allows the verifier to transfer funds if a contradiction is found in the prover's response, and the third part allows the prover to transfer all funds with their signature after a certain period.

If the verifier continues to doubt the prover, both parties can repeat the challenge and response transactions multiple times until the verifier is satisfied or all logic gate challenges are completed, after which the verifier accepts the prover's commitment as correct. Throughout this process, the verifier will keep challenging the prover, and the prover will respond with evidence to support the correctness of their binary circuit commitment. However, if a contradiction is found dur-

ing the challenge and response process, indicating potential malicious behavior by the prover, the verifier can immediately generate a new transaction to transfer the funds as a penalty to the prover.

8.2. Babylon

In January 2022, Fisher and Professor David Tse from Stanford University co-founded the Babylon project. This project aims to explore the sharing of Bitcoin's security [38, 39]. The vision of the project is to leverage Bitcoin's Proof-of-Work (PoW) security to enhance the security of other blockchains based on Proof-of-Stake (PoS) consensus mechanisms. In a PoS chain [40], participants secure the network by staking capital. To validate blocks, participants must lock a certain amount of tokens as collateral. This staking mechanism incentivizes participants to become validators, honestly validate blocks, and maintain blockchain security. When validators stake more funds, they have a stronger economic incentive to protect and support the network. As the staked amount increases, attacks become more expensive and difficult because attackers would need to acquire or rent more tokens to control the network. Thus, the staking mechanism in PoS systems strengthens the overall security of the network. In PoS-based blockchains like Ethereum, Cosmos, and BNB Chain, security is typically ensured by staking their native tokens (e.g., ETH, ATOM, BNB). However, relying solely on native tokens for security limits the PoS chain's

security to the total market value of its native tokens. If the total market value of the native tokens is low, attackers can more easily accumulate enough tokens to launch an attack, threatening network security. This limitation makes the security of PoS chains heavily dependent on the market value of their native tokens, and market volatility can reduce security. To address this issue, a method known as “cross-chain staking” was recently proposed [41]. In this approach, foreign assets remain stored on their original chain but are locked in staking contracts and delegated to specific validators on the target chain. The staked assets are only slashed when the validator commits a slashing offense.

The Babylon project, on the other hand, introduces a breakthrough Bitcoin staking protocol. This protocol allows Bitcoin holders to stake directly on PoS blockchains without intermediaries, bridges, third-party custody, or oracles. The Babylon project has developed two mechanisms for cross-chain staking: the “Bitcoin Timestamp Protocol” and the “Bitcoin Staking Protocol,” which are introduced as follows.

Bitcoin Timestamp Protocol [42]: This protocol generates concise and verifiable timestamps for PoS chain block data on the Bitcoin blockchain. The Bitcoin Timestamp Protocol effectively addresses the “Long-range Attack” security challenge faced by PoS chains [37]. In a blockchain, a Long-range Attack is a theoretical threat that can undermine the PoS consensus. Similar to a 51% attack on PoW blockchains [43], a Long-range Attack allows attackers to create a false chain by starting from the genesis block with a higher token balance than the forging balance and deceiving nodes by creating their own chain branch. Some PoS chains, including Ethereum, use social consensus mechanisms to counteract this attack. In this mechanism, stakeholders periodically make off-chain decisions about the correct block at the latest height and ignore any other potential forks. However, this approach relies on the subjective opinions of stakeholders, compromising decentralization. Moreover, social consensus takes time, so most PoS chains impose long unbonding periods (usually several weeks) to ensure validators cannot easily exit before consensus is reached. The Babylon project introduces the Bitcoin Timestamp Protocol, which timestamps PoS chain block data on the Bitcoin blockchain. In the event of a fork caused by a Long-range Attack, the blocks generated by the attack would have later timestamps on the Bitcoin blockchain compared to normal blocks. Nodes can easily detect these forged blocks, rendering the attack ineffective.

Bitcoin Staking Protocol [44]: This protocol allows Bitcoin assets to provide economic security to any decentralized system through trustless (self-custody) staking. The Babylon project uses a remote staking method. It locks the staked Bitcoin in a contract on the Bitcoin chain and slashes the staked Bitcoin on the Bitcoin chain when the staker violates the PoS chain protocol. However, Bitcoin’s scripting language has limited expressiveness and cannot support smart contracts. The Babylon project overcomes this challenge by combining advanced cryptographic techniques (extractable one-time signature), consensus protocol innovations, and optimized use of Bitcoin’s scripting language.

This enables the protocol to slash the staked BTC on the Bitcoin blockchain in case of a violation. The working principle of the Bitcoin Staking Protocol is as follows:

- **Stake Bitcoin:** The staker initiates a staking transaction on the Bitcoin blockchain. This transaction creates a UTXO with two spending conditions: 1) a time lock, allowing the staker to retrieve the staked funds after a certain period using their key, and 2) a condition allowing the validator to destroy this UTXO using a special “extractable one-time signature” (EOTS) in the event of a violation by the staker. If delegated, this EOTS belongs to the validator to whom the staker has delegated.
- **Validation on the PoS Chain:** Once the staking transaction is confirmed on the Bitcoin chain, the staker (or the validator to whom the staker has delegated) can start validating the PoS chain and sign votes for valid blocks using the EOTS key.

To implement these mechanisms, the Babylon project is architected as shown in the Figure 18, consisting of three parts:

1. **Bitcoin Blockchain:** In the Babylon project, the Bitcoin blockchain provides timestamping services to PoS chains, offering reliable timestamps for other blockchains. Additionally, Bitcoin serves as staked capital, leveraging its high market value to provide robust security for PoS chain networks.
2. **Babylon Blockchain:** This serves as an intermediary layer and the control plane. Babylon Chain is built on the Cosmos SDK and links Bitcoin and other PoS blockchains through the standard Inter-Blockchain Communication (IBC) protocol. Its primary function is to efficiently aggregate timestamps generated from PoS chain block data. This approach is necessary because the limited and expensive block space on the Bitcoin blockchain makes it unsustainable to individually and continuously generate timestamps for each PoS chain block on the Bitcoin blockchain.
3. **PoS Blockchains:** In the Babylon project, other PoS blockchains (such as Ethereum) act as security consumers. They use the Bitcoin security provided by Babylon Chain to enhance their own security. These PoS blockchains rely on the Proof-of-Stake mechanism to reach consensus and use staked capital as an economic incentive to protect and maintain network security.

In the Babylon project, participants can choose their roles based on their preferences, including Bitcoin holders, validator nodes, and delegators. Bitcoin holders can lock and stake their Bitcoin to provide economic security to the system, earning returns or rewards in the process. Validator

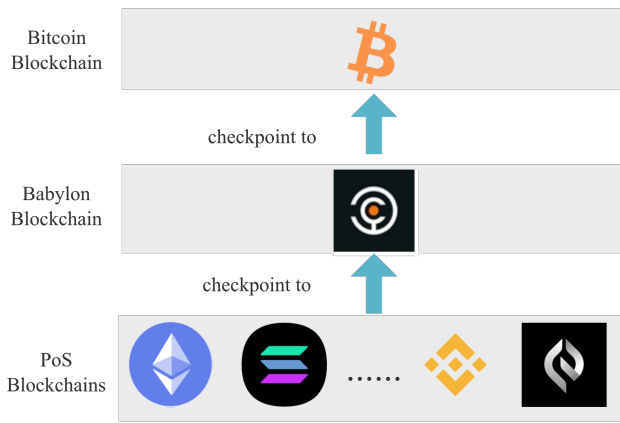


Figure 18: The architecture of the Babylon project.

nodes are responsible for verifying the legitimacy of transactions and blocks on the PoS chain. They need the technical ability and resources to perform these validation tasks. Delegators, who may lack the capacity or knowledge to run validator nodes, can delegate their Bitcoin to validators. By doing so, they transfer their voting power to the validator, who will then validate transactions and blocks on the PoS chain on their behalf.

The Bitcoin staking protocol in the Babylon project offers three key security properties. First, it provides fully slashable PoS security, meaning that if the integrity of the blockchain is compromised, one-third of the staked Bitcoin will be slashed. As long as two-thirds of the staked Bitcoin adheres to the PoS protocol, the PoS chain can maintain good liveness. Second, the security of stakers is assured—each Bitcoin staker can retrieve or unbond their staked Bitcoin at maturity, as long as they honestly follow the PoS protocol. Finally, the liquidity of stakers is guaranteed, allowing for rapid unstaking without requiring community consensus. A key design principle of the Bitcoin staking protocol is to make it as close as possible to the native staking mechanism of PoS chains, including PoS token holders, validator nodes, and delegators. For BTC staking, Bitcoin holders can become validators or delegate their Bitcoin to validators. This design allows stakers to participate in staking in a manner similar to PoS chains, thereby providing economic security.

8.3. BEVM

We classify the BEVM as a sidechain Bitcoin Layer 2 scaling solution. Due to the Taproot upgrade introducing the Schnorr signature algorithm, its linearity made multisignature schemes for Bitcoin feasible, offering a multisignature solution for Bitcoin's Layer 2 protocols without increasing script size. The Bitcoin-Ethereum Virtual Machine (BEVM), a Bitcoin EVM, will be launched in 2024 [45]. It is an EVM-compatible, fully decentralized Bitcoin Layer 2 protocol that uses BTC as gas. BEVM allows users to transfer their Bitcoin assets from the Bitcoin blockchain to the BEVM chain in a fully decentralized manner. Additionally, BEVM assets and data can be cross-chained back to the Bitcoin blockchain in a decentralized way. Since

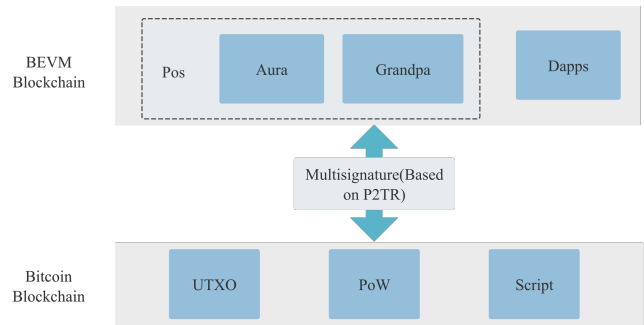


Figure 19: The architecture of BEVM.

BEVM is compatible with the Ethereum Virtual Machine (EVM) [46], Ethereum ecosystem decentralized applications (DApps) can run on Bitcoin's Layer 2, increasing BTC consumption scenarios and enabling decentralized Bitcoin financial services.

As shown in Figure 19, the BEVM chain uses the Proof of Stake (PoS) mechanism to generate blocks and employs the Aura consensus algorithm to elect block-producing nodes. Here's how it works: First, a group of PoS consensus nodes is elected based on the amount of stake they hold. These nodes, selected according to their stake, take turns generating blocks in a round-robin order. During block generation, the BEVM chain uses the Grandpa algorithm [47] to achieve BFT consensus. Grandpa finalizes consensus through validator votes, ensuring that the generated block is widely accepted by the network and becomes an irreversible part of the chain.

To transfer a user's Bitcoin assets from the Bitcoin blockchain to the BEVM chain, this solution synchronizes Bitcoin light node data on the BEVM chain. Validators on the BEVM chain verify the data provided by the Bitcoin light node client and reach consensus. Once consensus is achieved, the validated Bitcoin light node client data is synchronized into the EVM's underlying account system. The Bitcoin asset holder can then use their Bitcoin on EVM to interact with Ethereum smart contracts and decentralized applications (DApps).

To transfer assets from the BEVM chain back to the Bitcoin blockchain, a cross-chain transaction must be submitted on the EVM platform. First, the n PoS consensus nodes on BEVM perform a BFT vote using a BTC threshold custody contract, requiring more than two-thirds agreement. Once the vote passes, the threshold signature script is used as a leaf node in a MAST to generate a Taproot output key. This produces a Taproot transaction, which is then submitted to the Bitcoin blockchain, completing the cross-chain asset interaction. To spend this transaction, the threshold signature script must be used as an unlocking script to unlock the UTXO.

8.4. STACKS

Stacks was launched in early 2021 as a smart contract layer for Bitcoin, using the Proof of Transfer (PoX) consensus protocol [48]. As shown in Figure 20, Stacks allows Bitcoin transaction settlement and uses a secure contract language called Clarity, which can respond to Bitcoin transac-

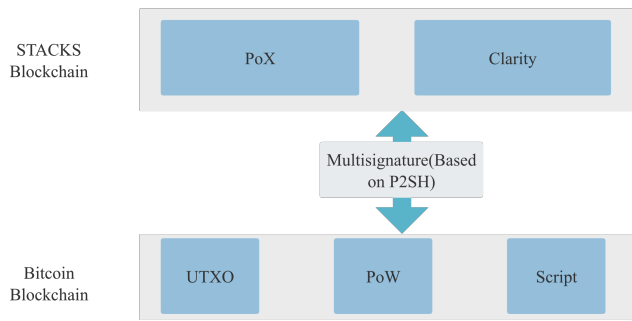


Figure 20: The architecture of STACKS.

tions. Additionally, Stacks supports atomic swaps between assets and BTC. The goal of Stacks is to transform Bitcoin from a passive asset into a productive one, supporting various decentralized applications to drive the growth of the Bitcoin economy. Similar to sidechains like RSK and Liquid, Stacks has its own global ledger and execution environment to support smart contracts without adding extra transaction burden to the Bitcoin blockchain. As shown in the figure, Stacks, like wBTC on Ethereum [49], RBTC on RSK[50], and L-BTC on Liquid [51], achieves asset pegging by delegating the peg to centralized custodians or a trusted and permissioned federation of entities that manage Bitcoin peg transactions using multisignature methods.

Despite some progress in providing a smart contract layer for Bitcoin and introducing innovative features, Stacks' centralized peg mechanism faces several issues:

1. **Dependence on Centralized Entities:** Stacks' centralized peg mechanism requires users to rely on centralized custodians or federated entities to manage the asset pegging and exchange process. This results in lower autonomy and control for users over their asset security and operations. Centralized entities may face security risks, misconduct, or single points of failure, posing potential threats to users' asset security.
2. **Trust and Transparency Issues:** The centralized peg mechanism requires users to trust that centralized entities will perform their duties correctly, with a lack of transparency and auditability. Users find it challenging to assess the fairness, compliance, and overall security of these entities' operations and decisions.
3. **Security Risks:** The centralized peg mechanism may face security risks. If centralized entities suffer from hacking, internal errors, or other security vulnerabilities, users' assets could be at risk of loss or theft. This poses a potential threat to asset security and may erode users' trust in the entire system.
4. **Contradiction with Decentralization:** Stacks' centralized peg mechanism contradicts the principle of decentralization. Although Stacks aims to provide more features and applications for Bitcoin, the centralized peg mechanism introduces centralized control and dependency.

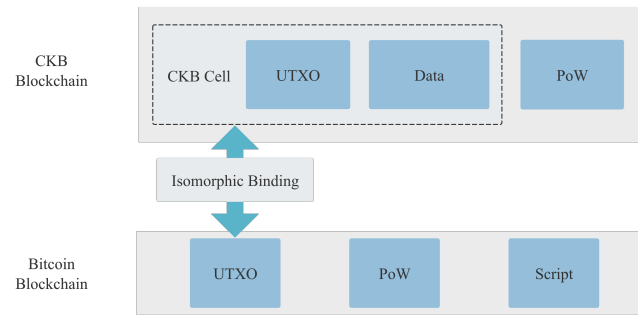


Figure 21: The architecture of RGB++.

Stacks' next major upgrade is the Nakamoto version, which primarily addresses the issue of centralized pegging of BTC assets in the original version by introducing a decentralized pegging solution that allows BTC to move in and out of the layer and be recorded on Bitcoin. Additionally, block production in Stacks will no longer be tied to miner elections, with miners producing blocks at a fixed rhythm. Since this upgrade requires a hard fork of the Stacks main chain, it needs approval from 70% of stackers, making the upgrade plan as challenging to implement as reorganizing the Bitcoin mainchain.

8.5. RGB++

Nervos Network is an open-source blockchain ecosystem project that offers a set of solutions to address scalability and interoperability issues in blockchain. As shown in Figure 21, the core of the Nervos Network project is the CKB (Common Knowledge Base) chain, a public blockchain that uses a Proof of Work (PoW) consensus mechanism and an improved UTXO model for storing and securing encrypted assets. The improved UTXO model is called the Cell model. Similar to Bitcoin's UTXO, a Cell in the CKB chain is also a transaction output. However, unlike Bitcoin's UTXO, which only represents a certain number of sats and cannot store additional data beyond the locking script, a Cell on the CKB chain not only contains cryptocurrency value but also has storage capacity that can be used to store any data. In the CKB chain, 1 CKB equals 1 Byte of storage space. Therefore, if a Cell contains 99 CKB tokens, that Cell will have 99 Bytes of on-chain storage space. This transforms the UTXO, which traditionally only stores an integer value representing token quantity, into a space capable of storing arbitrary data.

RGB++ is a protocol proposed by Cipher Wang, the co-founder of the Nervos Network project. This protocol provides a standard for issuing and managing assets on the Bitcoin blockchain. Notably, this protocol is integrated with the CKB chain as a Layer 2 solution for Bitcoin. Assets issued on the Bitcoin blockchain that comply with the RGB++ protocol (such as tokens, non-fungible tokens.) have their asset ledger managed on the CKB chain, where computation, verification, and data storage are handled. The RGB++ protocol, inspired by the RGB protocol, aims to enhance Bitcoin's performance by performing the computation, execution, and verification of token transactions off-chain, with final set-

tlement occurring on the Bitcoin chain. Additionally, the RGB++ protocol binds asset ownership to Bitcoin's UTXO, ensuring that the settlement of new assets occurs on the Bitcoin chain, meaning that operations such as issuing, trading, and transferring new assets must be completed by spending a Bitcoin UTXO.

RGB++ introduces a concept called Isomorphic Binding, which differs from the form of cross-chain bridges. Isomorphic Binding functions by binding UTXOs on the Bitcoin blockchain that meet the RGB++ protocol to the CKB chain's Cell, using the Cell as a "container" for RGB asset data and writing the key parameters of RGB assets into the Cell. Since RGB assets are bound to Bitcoin UTXOs, the logical form of the assets retains the characteristics of UTXOs.

It is worth mentioning that Cipher Wang, as the leader of the CELL Studio team, along with his team, developed a modular Bitcoin Layer 2 issuance platform called UTXO STACK. The Bitcoin Layer 2 chain issued by this platform is isomorphic with Bitcoin's UTXO model.

8.6. RGB

The RGB protocol, developed by the LNP/BP Standards Association, is a scalable and privacy-focused smart contract system for Bitcoin and the Lightning Network [52, 53]. The core technological components of RGB were proposed by Peter Todd in 2017, consisting of two main concepts: single-use-seals and client-side validation [54].

In RGB, tokens are associated with a Bitcoin UTXO (whether an existing UTXO or a temporarily created one). To transfer these tokens, the associated UTXO must be spent. When spending this UTXO, the Bitcoin transaction must include a commitment to a message containing the RGB payment information. This message defines the inputs, the UTXO to which the tokens will be sent, the asset ID, the amount, the spending transaction, and any other necessary data. Essentially, the core data running on RGB attaches itself to the Bitcoin blockchain via UTXO, leveraging the Bitcoin mainnet to secure the assets. However, this functionality faces two challenges:

1. **Verification Complexity:** Since the client-side validation of assets requires tracing every upstream UTXO of the asset, a significant amount of data verification is involved. The more times an asset is transferred, the greater the difficulty and cost of validation.
2. **Off-Chain Data Dependence:** Although assets can be verified, the Bitcoin blockchain only serves as a ledger for off-chain data. Bitcoin miners do not actually participate in the validation of RGB assets, necessitating third-party validation of the RGB ledger. Additionally, RGB smart contracts do not truly operate on-chain; each RGB-based smart contract is independent and cannot interact with others. For instance, if two tokens issued on RGB need to be swapped, this cannot be done directly like assets issued on the Ethereum Virtual Machine (EVM). Instead, the tokens must be transferred to the Lightning Network for interaction.

8.7. Lightning Network

We classify the Lightning Network as a state channel Bitcoin Layer 2 scaling solution. The Lightning Network was first proposed in a white paper written by Joseph Poon and Tadge Dryja in 2015. Its core idea is to use payment channels to avoid frequent interactions with the blockchain. Specifically, the Lightning Network allows participants to establish a two-way payment channel on the blockchain. Once the channel is established, participants can conduct multiple fast offline transactions within the channel. These transactions do not need to be recorded directly on the blockchain. Only when the final settlement is completed, the participants submit the transaction results to the blockchain. This greatly reduces the load and transaction costs of the Bitcoin. The emergence of the Lightning Network solves the scalability problem of the Bitcoin blockchain. Due to the slow creation speed of Bitcoin blocks and the limited number of transactions accommodated, transaction fees are high and transaction times are long. The use of the Lightning Network can improve the efficiency of block space usage, making small payments and high-frequency transactions more convenient and practical. The Lightning Network also has a high degree of privacy protection. The transaction information under the user chain will not spread on the network, and the transaction details are only visible to the parties involved. The underlying principles of the Lightning Network include multisignature addresses and hash time lock contracts (HTLC). Multisignature addresses allow users to specify the number of private keys required to pay funds and sign transactions. In the Lightning Network channel, participants lock funds through the 2-of-2 signature scheme, and two private keys are also required to transfer funds. A hashed timelock contract is a mechanism that enforces a contract between participants, allowing the other party to take remedial action and withdraw funds if one party does not play by the rules.

8.8. Comparison and Discussion of Limitations in Bitcoin Layer 2 Protocols

We presents a comparison of several representative Bitcoin Layer 2 protocols, focusing on their functionalities, architectures, and implementations. The comparison results are summarized in TABLE 3, which highlights key aspects such as protocol category, smart contract support, consensus mechanism, the technology of bridging, and programming language.

To begin with, in terms of protocol categories, BitVM stands as a representative of Rollup technology, combining off-chain computation with on-chain validation to significantly enhance transaction processing capacity. This makes it an important solution for addressing Bitcoin's scalability challenges. In contrast, protocols such as Babylon, BEVM, STACKS, and RGB++ are sidechain solutions that operate parallel to the Bitcoin mainchain, providing enhanced scalability while enriching the functionality and diversity of the Bitcoin ecosystem. Notably, the Lightning Network, as a representative of state channel technology, takes a different approach by leveraging existing blockchain resources with-

Table 3
Comparison of Layer2 Protocols

Protocol	Category	Smart contract	Consensus protocol	Bridging Technology	Language
BitVM	Rollup	Yes	-	-	Bitcoin script
Babylon	Sidechain	Yes	PoS	-	Go
BEVM	Sidechain	Yes	PoS	Multisignature(Based on P2TR)	Solidity
STACKS	Sidechain	Yes	PoX	Multisignature(Based on P2SH)	Clarity
RGB++	Sidechain	Yes	PoW	Isomorphic Binding	C
RGB	Client - side verification	Yes	-	-	Contractum
Lightning Network	State Channel	Not	-	-	-

out creating a new blockchain, streamlining transaction processes to achieve highly efficient payment handling.

The ability for smart contracts is a crucial factor in assessing the potential applications of Layer 2 protocols. BitVM, through its innovative Rollup architecture, extends Bitcoin's functionality to support complex smart contracts. However, BitVM is currently restricted to two-party interactions, limiting its applicability compared to multi-party or generalized verification protocols. Its challenge-response mechanism requires a potentially large number of on-chain transactions to resolve disputes. In complex cases, a full dispute may involve dozens or even hundreds of transactions on Bitcoin mainnet, resulting in a significant transaction fee burden, especially during network congestion. Babylon and BEVM, as sidechain protocols, embed smart contract support directly into their respective blockchain environments. Meanwhile, RGB utilizes the underlying architecture of the Lightning Network to enable indirect support for smart contracts.

Consensus mechanisms play a critical role in determining the level of decentralization and security of Bitcoin Layer 2 sidechains. Babylon and BEVM adopt Proof of Stake (PoS), encouraging active participation from network nodes through economic incentives, which ensures a high degree of decentralization. STACKS introduces the innovative Proof of Transfer (PoX) consensus, which has proven effective in maintaining network security and decentralization. RGB++, on the other hand, employs a Proof of Work (PoW) consensus, using complex mathematical problems to secure the network and guarantee its immutability.

When evaluating the security of sidechain protocols, the technology of bridging with the Bitcoin blockchain are crucial. The STACKS protocol employs a more traditional P2SH-based multisignature scheme to achieve cross-chain interactions with the Bitcoin blockchain. While this scheme effectively meets cross-chain requirements, it has significant limitations, as its multisignature mechanism supports only up to 15 participants. This limitation hinders broader node participation and reduces the decentralization of decision-making, which could impact the overall security and decentralization of the sidechain protocol. In contrast, the BEVM protocol demonstrates notable innovation in this regard by adopting a P2TR-based multisignature scheme as its bridging technology. This approach supports a larger number of participants (up to 1,000 participants within the protocol) to collaborate in the multisignature process. This significantly increases the participation breadth of nodes and the democ-

ratization of decision-making, thereby enhancing the security and decentralization of the network.

On the other hand, RGB++ employs a unique technology called Isomorphic Binding, which directly binds UTXOs from the Bitcoin blockchain, representing cross-chain assets in an alternative form. While technically distinct, this method focuses more on UTXO-level binding than on enhancing the security and decentralization of interactions through multisignature mechanisms. However, RGB++ still faces several inherent limitations. Specifically, RGB++ requires users to trace all upstream UTXOs for validation, leading to scalability bottlenecks. In addition, because the protocol depends on off-chain data storage, there is a risk that missing or unavailable data could undermine the security and usability of the system. Furthermore, the absence of a global state restricts interoperability and composability, making it difficult for smart contracts to interact or be integrated into more complex applications.

The Lightning Network enables rapid off-chain transactions through the use of bidirectional payment channels, with final settlement occurring on Bitcoin mainnet. While this design improves scalability and transaction throughput, it introduces several operational and security challenges.

First, channel liquidity management is a fundamental constraint: routing nodes must maintain balanced inbound and outbound capacity across their channels, which can be difficult to sustain under dynamic network conditions. Second, because routing requires nodes to remain online and capable of signing transactions in real time, hot wallet security becomes a critical concern—exposing funds to heightened risk of compromise. Third, the network relies on watchtower services to monitor the blockchain and intervene in the event of attempted channel breaches; this adds complexity and introduces new trust or availability assumptions.

Beyond these operational issues, the Lightning Network also faces structural centralization risks. As of recent measurements, a small number of highly connected nodes control a disproportionate share of the network's public capacity. For example, some studies [55] indicate that the top 10 nodes account for over 60% of total capacity, raising concerns about resilience and alignment with Bitcoin's broader decentralization goals.

9. Data Collection and Analysis

In order to scientifically evaluate the impact and actual results of the upgrade of Taproot proposal and its derived

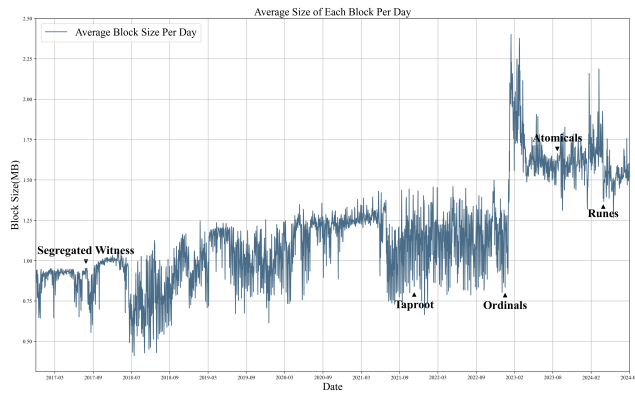


Figure 22: The average size of each block per day on the Bitcoin blockchain.

protocols (such as Ordinals, Atomicals) on the Bitcoin mainnet, we conducted data collection and analysis on the transactions on the Bitcoin blockchain. Specifically, we crawled all transactions on the Bitcoin blockchain from January 3, 2017, to August 21, 2024, and analyzed these transaction data to obtain the changes in the average block size per day, the usage trend of the P2TR transaction type, and the fluctuation of miner fees on the Bitcoin blockchain caused by Taproot-like transaction activities (especially those initiated by protocols such as Ordinals, Atomicals and Runes).

Figure 22 depicts the trend of the average block size per day on the Bitcoin blockchain from January 2017 to August 2024. It can be observed from the figure that after the two technical upgrades of 'Segregated Witness' and 'Taproot proposal' (on August 24, 2017, and November 14, 2021, respectively), the average block size per day showed an overall upward trend. Several factors contributed to this phenomenon. Firstly, key technological upgrades such as Seg-Wit and Taproot optimized transaction data structures, enhanced blockchain scalability, and reduced data redundancy, significantly increasing the effective capacity of transactions within blocks, allowing more transactions to be accommodated within the 1MB physical block size limit. Secondly, the introduction of new protocols based on Taproot (Ordinals, Atomicals, Runes, etc.) not only enriched the functionality and application scenarios of the Bitcoin mainnet but also had an impact on the block space requirements by introducing new data structures. These new protocols often involve more data elements or require more complex script execution, resulting in a larger block space occupied by a single transaction compared to traditional transactions. With the block size limit remaining the same, this means that when accommodating the same number of traditional transactions (measured by transaction count), more space is needed to store them due to the presence of complex transactions.

Figure 23 depicts the change in the percentage of P2TR transaction type in the total transaction types on the Bitcoin blockchain from January 2023 to August 2024. It can be seen from the figure that with the emergence of protocols such as Ordinals and Atomicals, P2TR transactions gradually occupy an important position among the various trans-

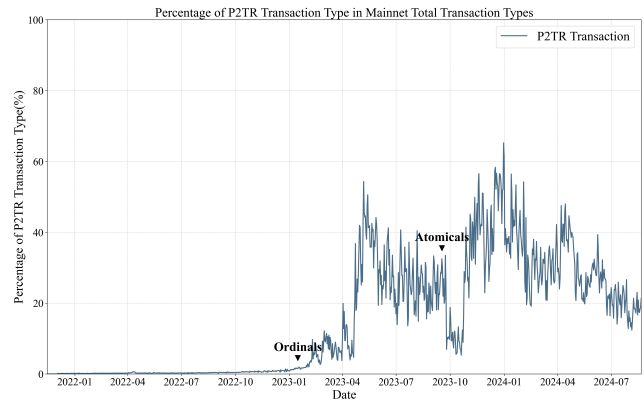


Figure 23: The percentage of P2TR transaction type on the Bitcoin blockchain.

action types on the Bitcoin blockchain (between approximately 20% and 50%). As mentioned earlier, the deploy, mint, and transfer operations of NFTs and fungible tokens in protocols such as Ordinals and Atomicals rely on the P2TR transaction type. The data in Figure 23 indicates the successful application of protocols such as Ordinals and Atomicals, which indeed greatly boost the proportion of P2TR transactions on the Bitcoin blockchain. The driving force behind this increase in P2TR transaction adoption stems from the launch of the Ordinals protocol on January 15, 2023, which pioneered data inscriptions using P2TR transaction to embed NFT metadata onto individual satoshis. This innovation ignited widespread market enthusiasm, propelling the percentage of P2TR transaction type to approximately 50% by March 2023. Subsequently, the Atomicals protocol was launched on September 17, 2023, further elevating the percentage of P2TR transaction type share to around 60% by late 2023, driven by the expanded use of P2TR transaction for token standards such as ARC-20 and the growing volume of token-related transactions.

Figure 24 and Figure 25 depicts the increase in transaction volumes and the transaction fees required for the deploy, mint, and transfer operations of fungible tokens in protocols such as BRC-20, ARC-20, and Runes on the Bitcoin blockchain from January 2023 to July 2024. It can be observed from the figure that these fungible token standards in Bitcoin have attracted significant market attention, leading to a large number of users participating in token deploy, mint, and transfer operations, thereby driving up transaction fees on the Bitcoin blockchain. The fees generated from these token operations become a substantial source of income for miners. The data clearly indicate that fee spikes are closely tied to surging transaction volumes.

The initial fee spike, reaching 200-250 BTC in April-May 2023, is directly linked to the Ordinals protocol launch on January 15, 2023. This protocol introduced inscriptions using P2TR transaction, embedding extensive data into blocks. This innovation triggered a rapid surge in user activity, as market enthusiasm drove large-scale minting and transfer of inscriptions, causing daily transaction volumes rising to approximately 600,000. As the number of trans-

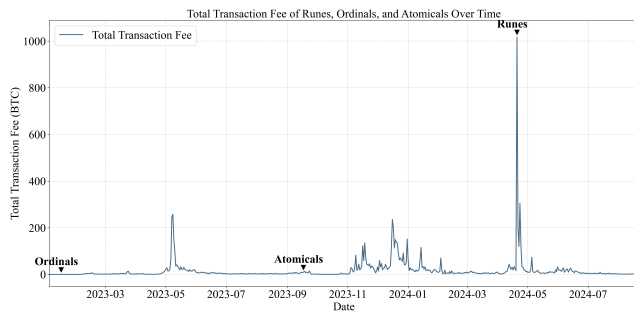


Figure 24: The transaction fees used by the BRC-20, ARC-20 and Runes operations.

actions exceeded the available block capacity, users competed by offering higher fees to prioritize their transactions for inclusion in a block. Heightened competition for block inclusion, driven by surging demand, limited block space, and the fixed block generation interval, was the primary cause of the sharp increase in transaction fees during this period. This effect was further amplified by miners' transaction selection mechanism, which prioritizes higher-fee transactions for block inclusion. Subsequently, the Atomicals protocol, launched on September 17, 2023, contributed to fee increases of 100-200 BTC from November 2023 to March 2024, as transaction volumes climbed to around 600,000 due to token-related activities. The widespread adoption of Ordinals and Atomicals protocols led to sustained high transaction volumes, which intensified competition for limited block space. The most fee surge occurred during the launch of the Runes protocol on April 20, 2024, when transaction volumes exceeded 800,000 per day, driving fees to a peak of 1000 BTC.

These trends highlight the ongoing scarcity of block space on Bitcoin, especially during major protocol launches. As competition intensified, low-fee transactions faced long delays as miners prioritized higher-fee transactions, and miner revenue increased dramatically. During periods of high demand, such as the Runes peak in April 2024, low-fee transactions experienced delays of several hours as they were deprioritized in favor of higher-fee transactions. Miners capitalized on this, experiencing a 300% revenue increase during the period. However, the elevated fees and extended wait times may have discouraged regular users, indicating a trade-off between protocol innovation and network accessibility that could influence long-term adoption. The combined analysis of Figures 24 and Figure 25 demonstrates how transaction volume surges, triggered by the Ordinals, Atomicals, and Runes launches, directly drive fee escalations and congestion, highlighting the urgent need to design and implement new scalability solutions that can mitigate network congestion and high transaction fees.

10. Conclusion

This comprehensive survey has elucidated the significant advancements in Bitcoin's Layer 1 and Layer 2 protocols, which have substantially enhanced its programming functionalities. The upgrade brought by the Taproot proposal in

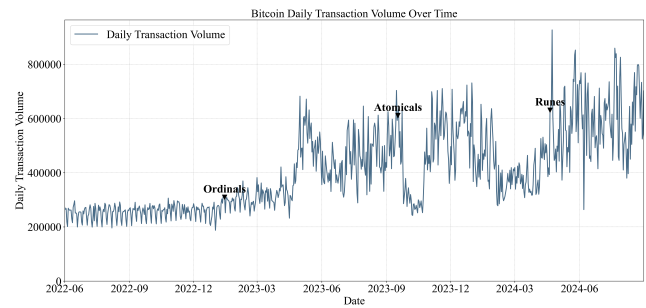


Figure 25: The daily transaction volume on the Bitcoin blockchain.

2021, introducing the Schnorr signature algorithm and P2TR transaction type, has catalyzed remarkable improvements in Bitcoin's privacy and programmability. This evolution has fostered the development of innovative protocols such as Ordinals, Atomicals, and BitVM, expanding Bitcoin's applications into realms previously thought impractical, including non-fungible tokens (NFTs) and more sophisticated fungible token standards (e.g., BRC-20, ARC-20). Our analysis of transaction data from the Bitcoin blockchain revealed positive trends, including increased block capacity and miner revenues driven by these new protocols. The findings affirm that these Layer 2 solutions not only address the scalability challenges that have historically impeded Bitcoin's growth but also open up new avenues for application development within the network. In conclusion, this survey provides a critical foundation for understanding Bitcoin's technological trajectory. We illuminate how Layer 1 and Layer 2 innovations are synergistically enhancing Bitcoin's capabilities, potentially catalyzing broader adoption and diversifying its applications.

Despite these advancements, several limitations remain. The integration of complex functionalities into Bitcoin's blockchain can lead to increased transaction sizes and potential network congestion. Moreover, several limitations in the broader Bitcoin ecosystem remain unresolved:

Dynamic Fee Market Mechanisms: Our empirical data demonstrates how transaction volume surges, triggered by the Ordinals, Atomicals, and Runes launches, directly drive fee escalations and congestion. This problem is compounded by Bitcoin's inherent protocol constraints, such as the 1MB block size limit and fixed block generation interval, which fundamentally restrict network throughput during periods of high demand. However, miners' transaction selection mechanism, which prioritizes higher-fee transactions for block inclusion, further amplifies these challenges by making it even more difficult for low-fee transactions to be confirmed in a timely manner. Future research should focus on designing more effective fee market mechanisms and transaction selection strategies to alleviate network congestion during periods of high demand. Potential directions include developing more adaptive fee market models, incentive-compatible transaction packing algorithms, and mempool management strategies that optimize throughput without undermining network security or decentralization. Ultimately, new de-

signs should aim to balance miner incentives with users' need for fair, efficient, and reliable transaction inclusion, especially during periods of network congestion.

Trust-Minimized Bridge Protocols: Cross-chain bridge exploits caused approximately \$2 billion in losses across 13 bridge hacks in 2022 alone, accounting for 69% of all crypto thefts that year [56]. These events underscore the urgent need for trust-minimized interoperability mechanisms that eliminate reliance on centralized validators. Future work should explore alternative bridge architectures that leverage dispute-driven verification, on-chain fraud proofs, or lightweight cross-chain proof systems to enhance security, transparency, and decentralization.

Decentralized Infrastructure for Token Protocols: While Taproot proposal enhanced the privacy of Bitcoin mainnet, token standards such as BRC-20 introduce critical centralization vulnerabilities through their dependence on off-chain indexers for state reconstruction. Relying on off-chain indexers introduces a trust dependency on centralized entities that fundamentally contradict Bitcoin's decentralized principles. Future research should develop: (1) Multi-party computation protocols that enable distributed indexing networks in which no single party controls the entire state reconstruction process, and that provide cryptographic guarantees of consistency across participating nodes. (2) Zero-knowledge proof systems allowing indexers to produce succinct, verifiable proofs that demonstrate the correctness of reconstructed token states, so that any participant can independently validate results without needing to trust the indexer or recompute all historical data.

Security and Scalability of Layer 2 Solutions: While the Lightning Network enables rapid, scalable off-chain transactions via bidirectional payment channels, it presents several operational and security challenges. Key limitations include: (1) channel liquidity constraints requiring continuous rebalancing, (2) heightened risk associated with hot wallets needed for real-time signing, and (3) reliance on watchtower services for fraud mitigation, which introduces new trust and availability assumptions. Moreover, the network exhibits a tendency toward centralization, with a small number of nodes controlling a disproportionate share of total capacity—potentially undermining the network's resilience and alignment with Bitcoin's decentralization principles. These challenges suggest that future research should focus on improving channel management protocols, enhancing hot wallet security, developing more robust and decentralized watchtower solutions, and mitigating structural centralization risks to preserve the Lightning Network's scalability and alignment with Bitcoin's foundational goals.

Overall, while the current protocols have enhanced Bitcoin's ecosystem, ongoing innovation and research are essential to address existing challenges and unlock further potential in decentralized finance and beyond.

CRedit authorship contribution statement

Guofu Liao: Methodology, Conceptualization, Investigation, Writing - Review & Editing.

Taotao Wang: Methodology, Conceptualization, Investigation, Writing - Review & Editing.

Long Shi: Review & Editing.

Jiqun Zhang: Review & Editing.

Yihan Xia: Investigation, Writing - Review & Editing.

Qing Yang: Writing - Review & Editing.

Xiaoxiao Wu: Review & Editing.

Shengli Zhang: supervision, Review & Editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was supported in part by the Guangdong Basic and Applied Basic Research Foundation (2024A1515012407), in part by the Shenzhen Science and Technology Program (JCYJ20220531101015033).

References

- [1] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008.
- [2] Andreas M Antonopoulos. *Mastering Bitcoin: unlocking digital cryptocurrencies*. " O'Reilly Media, Inc.", 2014.
- [3] James Lovejoy, Madars Virza, Cory Fields, Kevin Karwaski, Anders Brownworth, and Neha Narula. Hamilton: A {high-performance} transaction processor for central bank digital currencies. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 901–915, 2023.
- [4] Vitalik Buterin et al. Ethereum white paper. *GitHub repository*, 1(22-23):5–7, 2013.
- [5] Caspar Schwarz-Schilling, Joachim Neu, Barnabé Monnot, Aditya Asgaonkar, Ertem Nusret Tas, and David Tse. Three attacks on proof-of-stake ethereum. In *International Conference on Financial Cryptography and Data Security*, pages 560–576. Springer, 2022.
- [6] Nicolas Christin and Reihaneh Safavi-Naini. Financial cryptography and data security. In *18th International Conference*. Springer, 2014.
- [7] Jeremy Rubin, Manali Naik, and Nitya Subramanian. Merkelized abstract syntax trees. *XP055624837, Dec*, 16(3), 2014.
- [8] Joseph Poon and Thaddeus Dryja. The bitcoin lightning network: Scalable off-chain instant payments, 2016.
- [9] Pieter Wuille, Eric Lombrozo, and Johnson Lau. Bip-0141: Segregated witness (consensus layer). <https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki>, 2015.
- [10] Pieter Wuille and Andrew Poelstra. Bip-0340: Schnorr signatures for secp256k1. <https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki>, 2020.
- [11] Pieter Wuille, Jonas Nick, and Anthony Towns. Taproot: Segwit version 1 spending rules. *Bitcoin Improvement Proposal (BIP)*, 341, 2020.
- [12] Stefano Lande and Roberto Zunino. Sok: unraveling bitcoin smart contracts. *Principles of Security and Trust LNCS*, 10804:217, 2018.
- [13] Anubha Jain and Emmanuel S Pilli. Sok: Digital signatures and taproot transactions in bitcoin. In *International Conference on Information Systems Security*, pages 360–379. Springer, 2023.

- [14] Ningran Li, Minfeng Qi, Qin Wang, and Shiping Chen. Bitcoin inscriptions: Foundations and beyond. In *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 1–5. IEEE, 2024.
- [15] Anna Vacca, Andrea Di Sorbo, Corrado A Visaggio, and Gerardo Canfora. A systematic literature review of blockchain and smart contract development: Techniques, tools, and open challenges. *Journal of Systems and Software*, 174:110891, 2021.
- [16] Yajing Wang, Jingsha He, Nafei Zhu, Yuzi Yi, Qingqing Zhang, Hongyu Song, and Ruixin Xue. Security enhancement technologies for smart contracts in the blockchain: A survey. *Transactions on Emerging Telecommunications Technologies*, 32(12):e4341, 2021.
- [17] Ray Neiheiser, Gustavo Inácio, Luciana Rech, Carlos Montez, Miguel Matos, and Luis Rodrigues. Practical limitations of ethereum’s layer-2. *IEEE Access*, 11:8651–8662, 2023.
- [18] Monika Di Angelo and Gernot Salzer. A survey of tools for analyzing ethereum smart contracts. In *2019 IEEE international conference on decentralized applications and infrastructures (DAPPCON)*, pages 69–78. IEEE, 2019.
- [19] Xiangfan Wu, Ju Xing, and Xiaoqi Li. Exploring vulnerabilities and concerns in solana smart contracts. *arXiv preprint arXiv:2504.07419*, 2025.
- [20] Dechao Kong, Xiaoqi Li, and Wenkai Li. Characterizing the solana nft ecosystem. In *Companion Proceedings of the ACM Web Conference 2024*, pages 766–769, 2024.
- [21] Sergi Delgado-Segura, Cristina Pérez-Sola, Guillermo Navarro-Arribas, and Jordi Herrera-Joancomartí. Analysis of the bitcoin utxo set. In *International Conference on Financial Cryptography and Data Security*, pages 78–91. Springer, 2018.
- [22] Krzysztof Okupski. Bitcoin developer reference. *Eindhoven*, pages 3–4, 2014.
- [23] Binbing Hou and Feng Chen. A study on nine years of bitcoin transactions: Understanding real-world behaviors of bitcoin miners and users. In *2020 IEEE 40th international conference on distributed computing systems (ICDCS)*, pages 1031–1043. IEEE, 2020.
- [24] Pieter Wuille, Jonas Nick, and Anthony Towns. Bip-0342: Validation of taproot scripts. <https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki>, 2020.
- [25] Pieter Wuille. Bip-0032: Hierarchical deterministic wallets. <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>, 2012.
- [26] Gregory Neven, Nigel P Smart, and Bogdan Warinschi. Hash function requirements for schnorr signatures. *Journal of Mathematical Cryptology*, 3(1):69–87, 2009.
- [27] Felix Kleinwort, Wolf Posdorfer, and Janick Edinger. Analyzing the effect of taproot on bitcoin deanonymization. In *2023 IEEE 43rd International Conference on Distributed Computing Systems Workshops (ICDCSW)*, pages 25–30. IEEE, 2023.
- [28] Junfeng Xie, F Richard Yu, Tao Huang, Renchao Xie, Jiang Liu, and Yunjie Liu. A survey on the scalability of blockchain systems. *IEEE network*, 33(5):166–173, 2019.
- [29] Anthony Towns Pieter Wuille, Jonas Nick. Validation of taproot scripts, 2012.
- [30] Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. Simple schnorr multi-signatures with applications to bitcoin. *Designs, Codes and Cryptography*, 87(9):2139–2164, 2019.
- [31] Elizabeth Crites, Chelsea Komlo, and Mary Maller. How to prove schnorr assuming schnorr: Security of multi-and threshold signatures. *Cryptology ePrint Archive*, 2021.
- [32] Casey Rodarmor and Raph Japh. Ordinal theory handbook. <https://docs.ordinals.com/>, 2025.
- [33] Atomicals Protocol Developers. Atomicals documentation: User guide and developer reference. <https://docs.atomicals.xyz/>, 2025.
- [34] Cosimo Sguanci, Roberto Spatafora, and Andrea Mario Vergani. Layer 2 blockchain scaling: A survey. *arXiv preprint arXiv:2107.10881*, 2021.
- [35] Robin Linus. Bitvm: Compute anything on bitcoin. *URL: https://bitvm.org/bitvm.pdf-(12.12.2023)*, 2023.
- [36] Matthew Armstrong. Ethereum, smart contracts and the optimistic roll-up, 2021.
- [37] Benjamin Charles Moszkowski. *Reasoning about digital circuits*. Stanford University, 1983.
- [38] Babylon Labs. Babylon: Bitcoin-secured staking protocol. <https://babylonchain.io/>, 2024.
- [39] Ertem Nusret Tas, David Tse, Fangyu Gai, Sreeram Kannan, Mohammad Ali Maddah-Ali, and Fisher Yu. Bitcoin-enhanced proof-of-stake security: Possibilities and impossibilities. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 126–145. IEEE, 2023.
- [40] Fahad Saleh. Blockchain without waste: Proof-of-stake. *The Review of financial studies*, 34(3):1156–1190, 2021.
- [41] Osmosis Labs. Mesh security: Cross-chain economic security model for cosmos. <https://github.com/osmosis-labs/mesh-security>, 2023.
- [42] The Babylon Team. Bitcoin staking: Unlocking 21 m bitcoins to secure the proof-of-stake economy (litepaper). [https://docs.babylonlabs.io/papers/btc_staking_litepaper\(EN\).pdf](https://docs.babylonlabs.io/papers/btc_staking_litepaper(EN).pdf), 2023.
- [43] Long Shi, Taotao Wang, Jun Li, Shengli Zhang, and Song Guo. Pooling is not favorable: Decentralize mining power of pow blockchain using age-of-work. *IEEE Transactions on Cloud Computing*, 11(3):2756–2769, 2022.
- [44] Evangelos Deirmentzoglou, Georgios Papakriakopoulos, and Constantinos Patsakis. A survey on long-range attacks for proof of stake protocols. *IEEE access*, 7:28712–28725, 2019.
- [45] BEVM Foundation. Bevm — an evm-compatible bitcoin layer 2 white paper. <https://github.com/GEBcore/BEVM-white-paper-2023>, 2023.
- [46] Ethereum Community. Ethereum tests documentation. <https://ethereum-tests.readthedocs.io/en/latest/>, 2017.
- [47] Alistair Stewart and Eleftherios Kokoris-Kogia. Grandpa: a byzantine finality gadget. *arXiv preprint arXiv:2007.01560*, 2020.
- [48] Stacks Foundation. Stacks documentation. <https://docs.stacks.co/>, 2025.
- [49] Giulio Caldarelli. Wrapping trust for interoperability: A preliminary study of wrapped tokens. *Information*, 13(1):6, 2021.
- [50] Sergio Demian Lerner, Javier Álvarez Cid-Fuentes, Julian Len, Ramon Fernández-Valencia, Patricio Gallardo, Nicolás Vescovo, Raúl Laprida, Shreemoy Mishra, Federico Jinich, and Diego Masini. Rsk: A bitcoin sidechain with stateful smart-contracts. *Cryptology ePrint Archive*, 2022.
- [51] Liquid Network. Technical overview – the liquid network. <https://docs.liquid.net/docs/technical-overview>, 2023.
- [52] LNP/BP Standards Association and RGB Development Team. Rgb faq: Frequently asked questions about the rgb protocol. <https://www.rgbfaq.com/>, 2021.
- [53] Maxim Orlovsky, Peter Todd, Giacomo Zucco, Federico Tenga, and Olga Ukolova. Rgb blackpaper: Turing-complete, scalable & confidential smart contract layer for bitcoin & lightning network. <https://blackpaper.rgb.tech/>, 2023.
- [54] Peter Todd. Scalable semi-trustless asset transfer via single-use-seals and proof-of-publication. <https://peterodd.org/2017/scalable-single-use-seal-asset-transfer>, 2017.
- [55] Edward Gotham. Irrational economic action: Running a bitcoin lightning node for negative profit. *Ledger*, 8, 2023.
- [56] Chainalysis. Cross-chain bridge hacks highlight major security risks. <https://www.chainalysis.com/blog/cross-chain-bridge-hacks-2022/>, 2022.

Declaration of interests

- ☒ The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.
- ☐ The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

CRedit authorship contribution statement

Guofu Liao: Methodology, Conceptualization, Investigation, Writing - Review \& Editing.

Taotao Wang: Methodology, Conceptualization, Investigation, Writing - Review \& Editing.

Long Shi: Review \& Editing.

Jiqun Zhang: Review \& Editing.

Yihan Xia: Investigation, Writing - Review \& Editing.

Qing Yang: Writing - Review \& Editing.

Xiaoxiao Wu: Review \& Editing.

Shengli Zhang: supervision, Review \& Editing.